



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校 东南大学

参赛队号 24102860287

1.栾兴

队员姓名 2.徐泽辰

3.徐明晖

中国研究生创新实践系列大赛

“华为杯”第二十一届中国研究生

数学建模竞赛

题 目： WLAN 组网中网络吞吐量建模

摘 要：

无线局域网（WLAN）在现代通信中具有重要作用，而信道接入机制对网络的整体吞吐量有显著影响。本文针对三个关键问题进行系统的数据预处理与模型优化，探讨在高密度部署场景下影响系统吞吐的多种因素，旨在提升 WLAN 的整体性能。

在对**问题一**的分析中，数据预处理是确保建模准确性的基础。我们对实际 WLAN 网络的训练数据集进行了详细的清洗，包括异常值处理、缺失值填补和特征标准化等步骤。具体而言，数据集被分为 2AP 和 3AP 系统，以便分别建模，确保每组数据的特征能够被准确捕捉。通过随机森林回归模型的分析，我们得到了特征影响强弱排序，发现 AP 与 AP 之间的 RSSI 对发送机会的影响最大，2AP 系统中达到 87.96%，3AP 系统下达到 74.94%。其余影响因子强弱顺序依次：AP 和 STA 间的 RSSI、协议类型、网络拓扑、信号功率、NAV 门限值、BSS 编号。

在对**问题二**的分析中，我们重点分析信噪干扰比（SINR）与传输方式之间的关系，以预测在不同条件下 AP 的物理传输速率（PHY Rate）。我们发现，除了样本本身的特征数据，样本的 SINR 还与同一次实验的其他样本有关，因此在处理数据时，同时考虑了多行之间相互的影响。通过数据清洗和异常值检测，我们提高了数据的稳健性，并使用 Akima 插值拟合算法对 RSSI 测量值进行了统一处理，使其数量达到 540。我们选用的残差连接优化后的 CNN 模型预测(MCS,NSS)的验证集，构建的模型达到了 86.73%的准确率。

在对**问题三**的分析中，我们综合考虑了不同 AP 之间的干扰及丢包情况，构建了基于 CNN 的统一系统吞吐量预测模型，利用实测数据中的（MCS, NSS）组合作为输入特征。模型通过一维卷积进行特征提取，池化层减少特征维度，最终通过全连接层实现吞吐量的预测，并采用 Adam 优化器进行模型训练。在干扰情况下，理论吞吐量与仿真结果的误差控制在 5%以内，进一步验证了 RTS/CTS 机制在提升系统吞吐方面的有效性。我们对训练完成的预测模型，通过累积分布函数（CDF）进行模型精度的评估，模型在 2AP 和 3AP 系统中验证准确率超 95%，体现出该预测模型的优异性能和良好的泛化能力。

综上所述，本文通过系统的数据预处理与模型优化，成功分析了 WLAN 网络中多因素对吞吐量的影响，提出的解决方案为进一步提升 WLAN 性能提供了科学依据。研究结果对未来无线网络优化和新技术的应用具有重要的指导意义。

关键词：无线网络吞吐量；信道接入机制；随机森林回归；Akima 插值拟合算法；卷积神经网络；累积分布函数

目 录

一、问题重述	3
1.1 问题背景.....	3
1.2 问题重述.....	3
1.3 数据集概述.....	4
二、模型假设	6
2.1 基本假设.....	6
2.1.1 信道条件假设.....	6
2.1.2 节点行为假设.....	6
2.1.3 干扰假设.....	6
三、问题一的建模与求解.....	7
3.1 问题分析.....	7
3.2 模型构建与求解.....	7
3.2.1 数据预处理.....	7
3.2.2 分析参数.....	10
3.2.3 绘制特征相关性热力图.....	10
3.2.4 构建 AP 发送机会的预测模型.....	12
3.2.5 模型求解.....	13
3.2.4 结果分析.....	14
四、问题二的建模与求解.....	16
4.1 问题分析.....	16
4.2 模型构建与求解.....	17
4.2.1 数据预处理.....	17
4.2.2 模型选择和优化.....	20
4.2.3 模型求解.....	22
4.2.4 结果分析.....	23
五、问题三的建模与求解.....	24
5.1 问题分析.....	24
5.2 模型构建与求解.....	25
5.2.1 数据预处理.....	25
5.2.2 网络吞吐量预测模型.....	26
5.2.3 模型求解.....	27
5.2.4 结果分析.....	28
六、模型的改进与结论.....	30
6.1 模型改进建议.....	30
6.2 主要结论.....	30
6.3 研究贡献.....	31
参考文献	33
附录	34

一、问题重述

1.1 问题背景

无线局域网（WLAN）作为现代通信的核心，广泛应用于办公、教育和医疗等领域。然而，在高密度部署场景下，节点数量激增和小区覆盖重叠，导致干扰和碰撞加剧，实际带宽和数据传输速率显著下降。因此，准确预测吞吐量对于提升 WLAN 系统的稳定性和性能至关重要。

近年来，随着机器学习技术的发展，该技术在吞吐量预测中的应用逐渐增多。通过提取信道带宽、业务流量、发射功率、信道接入机制等特征，结合 RSSI、信干噪比（SINR）以及节点动态位置等信息，可以使用机器学习模型来预测系统吞吐量。目前的吞吐量预测方法大多基于仿真数据，但仿真结果通常无法全面反映实际环境中的信道条件、干扰动态以及复杂的业务流量，这导致了预测模型的精度有限，难以在真实部署中有效应用。

为应对这一挑战，本研究基于 WLAN 的实测数据，重点分析网络拓扑、节点间 RSSI、信道接入机制和干扰等因素对系统吞吐量的影响，力求构建一个更为精确的吞吐量预测模型。通过该模型，未来能够对 WLAN 系统进行优化，从而在多个场景中为用户提供便利。

1.2 问题重述

WLAN 网络的系统吞吐量是衡量网络性能的关键指标，影响其性能的主要因素包括节点间的 RSSI、信道竞争接入机制、CCA 门限、NAV 机制等。在 WLAN 网络中，AP 和 STA 的发送机会、数据传输速率（PHY Rate）、干扰情况等都会影响整体的吞吐量表现。本研究将针对三个关键问题，对 WLAN 网络的相关参数进行分析建模，探索不同情况下对系统吞吐量的预测方法。

问题一：AP 发送机会的影响分析

在 WLAN 网络中，AP 的发送机会受网络拓扑、业务流量、节点间 RSSI 和 CCA 门限等因素影响。AP 与 STA 的相对位置决定了 RSSI，业务流量大小影响信道竞争，CCA 门限则判断信道是否空闲。根据这些信息可建立模型，分析各因素对 AP 发送机会的影响，预测 AP 发送数据帧的总时长，并通过分类或统一建模提高预测精度。

问题二：AP 发送速率的建模

AP 发送速率由调制编码方案（MCS, NSS）决定，受节点间 RSSI 和信道干扰影响。通过分析 RSSI 和 CCA 门限信息，发现 AP 在不同传输阶段的选择不同，RSSI 较大时干扰增加，STA 信噪比下降。基于 RSSI 和门限信息可建立模型，预测 AP 最常用的（MCS, NSS），进而估算 AP 发送速率。

问题三：系统吞吐量的预测模型

WLAN 系统吞吐量受 AP 发送机会、PHY 速率、信道干扰等因素影响。基于网络拓扑、RSSI、信道门限信息，可构建综合模型预测吞吐量。因 RSSI 信息难完全反映信道变化，使用实测数据中的真实数据帧（MCS, NSS）作为输入变量，通过分类或统一建模方法，提高预测精度，为 WLAN 网络优化提供参考。

1.3 数据集概述

题目提供了多个数据集文件，包括 13 个训练集文件和 4 个测试集文件。不同的文件代表着不同 AP 系统在不同网络拓扑下以不同的门限值进行实验测得的实验数据。对数据集的简单观察有助于帮助建立对题目的整体理解，并且能够辅助产生处理数据的思路。

表 1-1 数据集分析

表格名称	样本数量
training_set_2ap_loc0_nav82	82
training_set_2ap_loc0_nav86	80
training_set_2ap_loc1_nav82	78
training_set_2ap_loc1_nav86	74
training_set_2ap_loc2_nav82	80
training_set_3ap_loc30_nav82	123
training_set_3ap_loc30_nav86	120
training_set_3ap_loc31_nav82	126
training_set_3ap_loc31_nav86	108
training_set_3ap_loc32_nav82	111
training_set_3ap_loc32_nav86	60
training_set_3ap_loc33_nav82	111
training_set_3ap_loc33_nav88	99
test_set_1_2ap	80
test_set_1_3ap	105
test_set_2_2ap	64
test_set_2_3ap	87
总计	1588

表 1-2 数据信息分析

数据列	含义
<i>test_id</i>	测试编号
<i>test_dur</i>	一次测试的时间
<i>loc</i>	AP 和 STA 部署在某一指定位置的编号
<i>bss_id</i>	同一拓扑里的 BSS 编号
<i>ap_name</i>	AP 的名字
<i>ap_id</i>	AP 编号
<i>sta_mac</i>	STA 的 MAC 地址
<i>sta_id</i>	STA 编号
<i>protocol</i>	AP 发送的流量类型，UDP 或 TCP
<i>pkt_len</i>	聚合前的数据长度
<i>pd</i>	包检测门限
<i>ed</i>	能量检测门限
<i>nav</i>	NAV 门限
<i>eirp</i>	AP 的发送功率

<i>ap_from_ap_x_sum_ant_rssi</i>	AP 各天线检测到来自 <i>ap_id</i> 为 AP_x 的 RSSI 的和, AP 配 8 根不同角度的天线接收信号, 取 8 根天线所测 RSSI 之和
<i>ap_from_ap_x_max_ant_rssi</i>	AP 各天线检测到来自 <i>ap_id</i> 为 AP_x 的 RSSI 的最大值, 每次记录各天线 RSSI 中的最大值, 该值用于 PD 门限和 ED 门限判决
<i>ap_from_ap_x_mean_ant_rssi</i>	AP 各天线检测到来自 <i>ap_id</i> 为 AP_x 的 RSSI 的平均值, 用于 NAV 门限判决
<i>sta_to_ap_x_sum_ant_rssi</i>	AP 各天线检测到来自 <i>sta_id</i> 为 STA_x 的 RSSI 的和
<i>sta_to_ap_x_max_ant_rssi</i>	AP 各天线检测到来自 <i>sta_id</i> 为 STA_x 的 RSSI 的最大值
<i>sta_to_ap_x_mean_ant_rssi</i>	AP 各天线检测到来自 <i>sta_id</i> 为 STA_x 的 RSSI 的平均值
<i>sta_from_ap_x_sum_ant_rssi</i>	STA 各天线收到的来自 <i>ap_id</i> 为 AP_x 的 RSSI 之和, STA 配 2 根不同角度的天线接收信号, 取 2 根天线所测 RSSI 之和
<i>sta_from_ap_x_max_ant_rssi</i>	STA 各天线收到的来自 <i>ap_id</i> 为 AP_x 的 RSSI 的最大值
<i>sta_from_ap_x_mean_ant_rssi</i>	STA 各天线收到的来自 <i>ap_id</i> 为 AP_x 的 RSSI 的平均值
<i>sta_from_sta_x_rssi</i>	STA 各天线收到的来自 <i>sta_id</i> 为 STA_x 的 RSSI 的和
<i>nss</i>	空间流数
<i>mcs</i>	MCS 阶数
<i>per</i>	丢包率。选用该 (MCS, NSS) 发送数据帧的失败个数与总个数的百分比
<i>num_ppdu</i>	一个数据帧的聚合个数。一次测试里, 统计每个数据帧的聚合个数, 取平均值
<i>ppdu_dur</i>	一个数据帧的时长。一次测试里, 统计每个数据帧的时长, 取平均值
<i>seq_time</i>	统计 AP 发送数据的帧序列的总时长
<i>other_air_time</i>	AP 除发送数据帧外, 发送管理帧等其他帧和接收所占用的时间
<i>throughput</i>	实测每个 AP 的吞吐量, 即每个 AP 单位时间内成功传输的比特数
<i>predict_throughput</i>	模型预测的每个 AP 的吞吐量
<i>error</i>	吞吐量预测误差, 即模型预测的吞吐量与实测吞吐量之差与实测吞吐量的百分比

二、模型假设

2.1 基本假设

根据提供的实测数据集，模型假设应基于实际观测数据的特征与条件进行设定。结合训练集的网络拓扑、RSSI、业务流量等关键要素，以下是针对模型假设的分析和总结：

2.1.1 信道条件假设

(1) 短时间内信道条件相对稳定

基于实测的 RSSI 数据，我们可以假设在短时间内信道的波动幅度较小，尤其是 RSSI 能够反映大尺度的信道变化。因此，数据建模时可以认为 RSSI 在短时间内不会出现剧烈波动，适合进行吞吐量的预测。

(2) 环境噪声和干扰相对可控

通过实测数据中的信号强度与干扰评估，假设环境噪声及干扰在短期内影响有限，信道模型中可忽略小规模噪声变化。

2.1.2 节点行为假设

(1) 节点按照标准协议通信

AP 和 STA 之间遵循 IEEE 802.11 协议的标准通信机制，包括使用 CCA 机制检查信道是否空闲、采用 NAV 机制协调信道访问，确保模型能够基于这些行为预测各节点的吞吐量。

(2) 基于实测 RSSI 的干扰处理

节点间的 RSSI 数据将作为评估干扰程度的基础，假设节点能够根据实际信号强度调整通信速率（MCS）与空间流数（NSS）。

2.1.3 干扰假设

(1) 干扰源以邻近 AP 为主

从训练数据中的 AP 间 RSSI 值来看，邻近 AP 是主要的干扰源。我们假设干扰主要来自信道复用的 AP，且干扰强度与 RSSI 直接相关。

(2) 多路径传播影响可忽略

由于多根天线的 RSSI 值已经在数据中汇总处理，多路径传播效应在模型中不再作为主要考虑因素。

三、问题一的建模与求解

3.1 问题分析

问题一的核心是通过分析 WLAN 网络的实际训练数据集，确定各个因素对 AP 发送机会的影响程度，并基于这些因素构建一个定量模型。这个模型的输出是 AP 在特定网络拓扑和干扰环境下的发送机会，而发送机会与 AP 在网络中的位置、流量类型、干扰强度等密切相关^[1]。

(1) 网络拓扑

AP 与 STA 之间的物理位置关系决定了节点之间的信号强度（RSSI），该强度直接影响 AP 与 STA 之间的通信质量。拓扑中的 AP 之间的相对位置决定了干扰的严重程度，特别是在同频 AP 较为密集的场景中，信道竞争更为激烈，从而减少 AP 的发送机会。因此，网络拓扑中的节点分布对 AP 发送机会的影响十分显著。

(2) RSSI（接收信号强度指示）

较高的 RSSI 通常表示更好的通信质量，AP 的发送机会也随之增加。然而，在高密度部署场景中，较低的 RSSI 可能导致更多的信号干扰，从而降低 AP 的发送成功率。RSSI 的波动对干扰强度以及发送成功率有直接影响，因此在建模过程中，RSSI 是不可忽视的因素。

(3) 业务流量的类型和规模

WLAN 中的业务流量通常分为 UDP 和 TCP 两种。UDP 协议是无连接的，数据包无需确认便可发送，因此 AP 的发送机会较大；而 TCP 协议则需要数据包发送后进行确认，这增加了发送过程中的等待时间，特别是在高负载情况下，AP 的发送机会会由于频繁的 ACK 等待而减少。此外，业务流量的大小也直接影响信道的占用时间，较大的数据包会延长 AP 的发送时间，从而减少其他节点获取信道的机会。

(4) 门限参数（包括 PD、ED、NAV 等）

PD 和 ED 门限通过判断信道是否空闲来决定 AP 是否能够发送数据，较低的门限值意味着 AP 更容易感知到其他设备的信号干扰，进而减少发送数据的机会。此外，NAV 门限通过设置静默期来限制其他节点使用信道，NAV 静默期越长，AP 的发送机会越少。

问题一的目标是分析各参数对 AP 发送机会的影响程度，并预测每个 AP 的发送机会，即发送数据帧序列的总时长。通过观察训练集，可以发现其中包含大量实验测量数据，因此首先需要对训练集进行数据清洗，排除异常值，填补缺失值，并删除明显无关的特征；接着对数据进行处理，提取与发送数据帧序列总时长存在显著关联的特征；然后将发送数据帧序列的总时长设定为预测目标，建立随机森林模型进行求解；最后，利用该模型确定各参数的影响强弱顺序，并对测试集中的样本进行发送数据帧序列总时长的预测。

3.2 模型构建与求解

3.2.1 数据预处理

在本研究中，对 WLAN 网络的实际训练数据集进行了系统的预处理，以确保后续建模过程的准确性和有效性。数据预处理步骤主要涉及数据清洗、异常值处理、缺失值填补、特征标准化、单位转换等环节。以下为数据预处理的步骤（以更复杂的 3AP 系统为例）。

(1) 数据集分组

由于 2AP 和 3AP 数据的特征数量存在差异，直接将它们合并建模可能导致模型无法准确捕捉各特征之间的关系。因此，为了确保模型能够充分利用特征信息，必须对这两组数据进行分开处理。通过对 2AP 和 3AP 数据的特征进行单独分析和建模，可以更准确地反映不同同频 AP 数量对发送机会的影响，从而提高预测的精度。因此，最终选择对 2AP 和 3AP 数据分别建模，以确保模型的有效性和准确性。问题一提供的数据集已根据文件名预先分为 2AP 实验数据和 3AP 实验数据，因此只需分别将 2AP 和 3AP 的数据文件合并，生成完整的 2AP 和 3AP 数据集即可。

(2) 数据清洗

通过观察发现，数据集中的样本存在异常值，且部分特征与问题求解无关。为了确保后续模型训练的准确性和有效性，必须对数据集进行清洗操作，清洗操作主要包括样本异常数据清理、无关特征清理和实验中完全一致特征清理。

a) 样本异常数据清理

由于实验测试误差、仪器问题等外部因素，实验数据中可能存在异常情况，因此需要对初步分析中明显错误的样本进行删除处理。问题一的数据集中存在与规定存储格式不一致的样本，为了避免对模型训练效果产生不利影响，必须提前将这些样本予以删除。

b) 无关特征清理

实验数据集中存在部分与目标无关的特征，为了提高训练效率和防止影响训练效果，需在数据预处理阶段对这些特征进行删除：

test_id: 表示在每组实验中不同实验次数的序号，由于存在多组实验，导致该序号在数据集中出现多次重复，可能会使模型在训练过程中建立错误关联。鉴于实验序号与实验结果显然无关，因此可以直接删除该特征。

num_ampdu, ppdu_dur, other_air_time: 分别表示 MAC 协议数据单元聚合个数、一个数据帧的时长、AP 除发送数据帧外发送管理帧等其他帧和接收所占用的时间。这三个特征与问题一的求解无关，因此删除。

c) 实验中完全一致特征清理

实验数据集中的部分特征存在一致性情况，需要经过具体分析选择处理策略：

test_dur, pkt_len: 表示一次测试的时间和数据长度。经过观察发现，在所有数据集（包括训练集和测试集）中，这两个特征的数值始终保持不变。为了提高训练效率，决定在数据处理阶段将其删除。

bss_id, ap_name, ap_mac, ap_id, sta_mac, sta_id: 通过观察可知，这 6 个特征在数据集中始终保持一一对应。为了提高训练效率，决定只保留 *bss_id* 特征，其余特征直接删除。

(3) 数据编码

数据集中有部分特征以字符形式表示。为了便于模型训练，需要将这些特征转换为数值型，因此将对这些特征进行 one-hot 编码处理：*loc_id, protocol, bss_id*。

(4) 单位转换

数据集中存在多个与信号功率有关的数据，单位均为 dBm。为了便于后续求和等操作的计算，需要根据以下转换公式将单位转换为线性单位。

$$P_{mW} = 10^{\frac{P_{dBm}}{10}} \quad (3-1)$$

其中， P_{mW} 是功率单位为 *mW* 时的数值， P_{dBm} 是功率单位为 *dBm* 时的数值。

(5) RSSI 数据处理

数据集中的 RSSI 数据多数都为长度不一的列表，代表着多次测量的数值。为了便于模型的训练，需要将其按照一定规则转换为数值型数据。

对于 RSSI 平均值（mean）数据，根据题目所述，AP 所有天线接受的 RSSI 平均值用于 NAV 门限判决，因此我们将每一个测量值与 NAV 门限的数值进行对比，计算超过 NAV 门限的测量值个数的占比 P_{over_nav} ：

$$\begin{cases} I_{NAV}(X) = \begin{cases} 0, X < P_{NAV} \\ 1, X \geq P_{NAV} \end{cases} \\ P_{over_nav} = \frac{\sum_{i=1}^N I_{NAV}(RSSI_{mean,i})}{N} \end{cases} \quad (3-2)$$

其中， $I_{NAV}(X)$ 是 NAV 门限判决函数， P_{NAV} 是 NAV 门限值， $RSSI_{mean,i}$ 是 RSSI 平均值的第 i 个测量值， N 是总共测量值的数目。

对于 RSSI 最大值（max）数据，它们主要用于 CCA 门限的判决，因此我们将测量值分别与 PD 门限和 ED 门限对比，分别计算占比并作为新的两个特征。

$$\begin{cases} I_{PD}(X) = \begin{cases} 0, X < P_{PD} \\ 1, X \geq P_{PD} \end{cases} \\ P_{PD} = \frac{\sum_{i=1}^N I_{PD}(RSSI_{max,i})}{N} \end{cases} \quad (3-3)$$

$$\begin{cases} I_{ED}(X) = \begin{cases} 0, X < P_{ED} \\ 1, X \geq P_{ED} \end{cases} \\ P_{ED} = \frac{\sum_{i=1}^N I_{ED}(RSSI_{max,i})}{N} \end{cases} \quad (3-4)$$

其中， $I_{PD}(X)$ 是 PD 门限判决函数， P_{PD} 是 PD 门限值， $I_{ED}(X)$ 是 ED 门限判决函数， P_{ED} 是 ED 门限值， $RSSI_{max,i}$ 是 RSSI 最大值的第 i 个测量值， N 是总共测量值的数目。

对于 RSSI 之和（sum）数据，我们认为低于 PD 门限的数值与噪声关系较为密切，因此将其与 PD 门限对比后，保留小于 PD 门限的数据求平均，得到 P_{noise} ：

$$\begin{cases} P_{noise,i} = \begin{cases} RSSI_{max,i}, RSSI_{sum,i} < P_{PD} \\ 0, RSSI_{sum,i} \geq P_{PD} \end{cases} \\ P_{noise} = \frac{\sum_{i=1}^N P_{noise,i}}{\sum_{i=1}^N I_{PD}(RSSI_{sum,i})} \end{cases} \quad (3-5)$$

其中， $I_{PD}(X)$ 是 PD 门限判决函数， P_{PD} 是 PD 门限值， P_{noise} 是 AP 不发送数据时的环境噪声功率， $RSSI_{sum,i}$ 是 RSSI 之和的第 i 个测量值， N 是总共测量值的数目。

(6) 数据标准化

在机器学习模型中，数据标准化是一个关键的预处理步骤，尤其在输入特征值的量级差异较大时。标准化的主要目的是消除不同特征之间的量纲差异，使模型能够更有效地进行学习，避免某些特征由于数值较大而对模型有过多的影响。未标准化的数据可能导致模型收敛缓慢、优化不稳定，或使某些特征的重要性被低估。标准化后，特征的均值为 0，标准差为 1，这样每个特征都被放置在相同的尺度上，有助于模型更好地捕捉数据的模式。

针对本题的数据集，使用以下的规则进行标准化：

$$X_{standard} = \frac{X - \mu}{\sigma} \quad (3-6)$$

其中， $X_{standard}$ 是标准化后的特征值， X 是原始特征值， μ 是特征的均值， σ 是特征的标准差。

通过上述预处理，原始数据集的问题得到了有效解决，同时数据被标准化后具备了一致性，方便后续模型的训练和测试。特征选择也提升了模型的训练效率和准确。

3.2.2 分析参数

基于上述分析，问题一中的模型将会构建一个框架，用数学公式描述 AP 发送机会与网络拓扑、业务流量、门限参数及 RSSI 等多个影响因子之间的关系。通过这些公式，能够有效预测 AP 的发送机会，并为后续的系统吞吐量预测提供基础。

1) RSSI (Received Signal Strength Indication)

RSSI 是衡量信号强度的指标。信号强度越高，成功发送的机会越大，较高的 RSSI 与较低的干扰相关，增加了 AP 发送的机会。此外，BSS_ID 列的数据表明信号的源 AP 信息。

2) 业务流量类型 (Protocol type)

数据传输使用的协议类型 protocol 会影响数据包的发送方式和频率，从而影响 AP 的发送机会。

3) AP 的发送功率

AP 的发送功率高低会影响信号的传输质量，同时也会改变对其他 AP 发送信号的干扰，因此总体来说会影响 AP 的发送机会。

4) CCA 门限 (Clear Channel Assessment Thresholds)

CCA 分为包检测门限 (PD) 和能量检测门限 (ED)，对应列分别为 pd 和 ed。当 RSSI 高于 PD 时，AP 能够侦听到信号并作出响应。若低于 ED，信道被判定为繁忙，AP 则不能发送数据。因此，PD 和 ED 门限的关系直接影响 AP 的发送机会。

5) NAV 门限 (Network Allocation Vector)

NAV 通过 nav 列表示，指示在特定时间段内其他节点不应发送数据。如果 AP 正在使用信道，周围的节点必须静默，这将影响 AP 的发送机会。

6) 网络拓扑

WLAN 网络的拓扑结构会对节点之间信号的干扰造成影响，从而影响 AP 的发送机会。在本题中，网络拓扑的差异体现在 loc_id 的不同。

为了预测 AP 的发送数据帧序列总时长 (seq_time)，我们设定了一个函数 f 预测每个 AP 的发送机会，函数公式如下

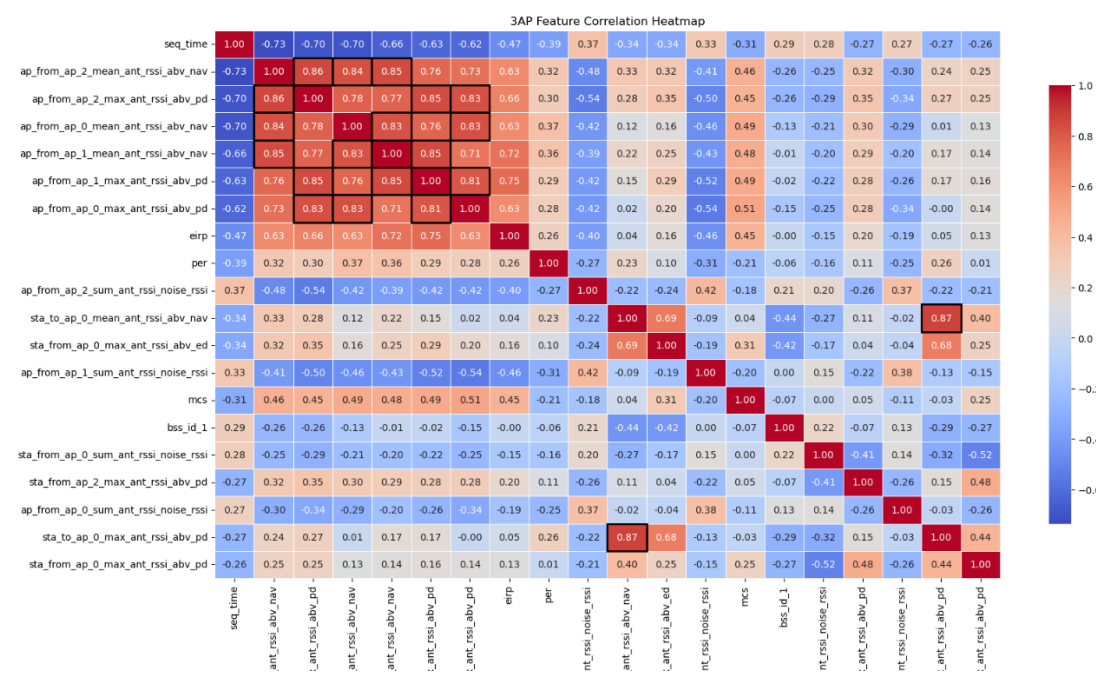
$$seqtime = f(RSSI, eirp, CCA, NAV, Packet_{time}, Protocol, locid) \quad (3-7)$$

3.2.3 绘制特征相关性热力图

为了对数据集的特征之间的相关性有基本的认知，需要先对数据集的各种特征做相关性分析。分别对 2AP 和 3AP 的特征做关联性分析，并且绘制热力图如图 3-1、图 3-2 所示。



图 3-1 2AP 系统特征相关性热力图



3.2.4 构建 AP 发送机会的预测模型

我们选择了随机森林回归模型用于确认影响因素的强弱顺序和预测 AP 的发送机会。

在无线网络中，AP 发送机会受多种因素影响（如 RSSI、CCA 门限、NAV、竞争窗口等），随机森林在分析影响因素的轻重顺序方面表现优越，它能够通过特征重要性度量来评估每个输入变量对预测结果的贡献大小。此外，随机森林的非线性建模能力使其能准确预测复杂环境下 AP 的发送机会，特别是在存在多个交互因素（如 RSSI、CCA 门限、NAV、竞争窗口和业务流量类型）的时候，随机森林能够通过处理大量不同样本的树，捕捉这些多变量间的复杂关系，并通过对所有树的预测结果进行加权平均，得出更精确的预测结果。

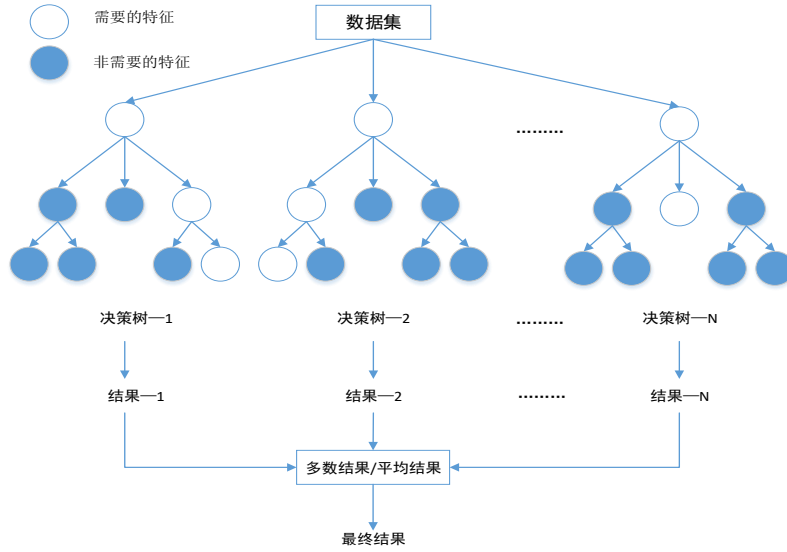


图 3-3 随机森林回归模型

(1) 模型公式

在传统的随机森林模型中，回归树的每个叶节点都输出一个预测值，而最终的预测结果是所有树预测值的平均值。具体公式如下：

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N f_i(x) \quad (3-8)$$

其中， $\hat{y}$ 是最终的预测值， N 为构建的树的数量， $f_i(x)$ 是第 i 棵树对输入 x 的预测值。

每棵决策树的构建基于决策树算法，它通过最大化信息增益来选择最佳分裂点。对于回归问题，通常使用均方误差 MSE 来评估分裂效果，公式为：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3-9)$$

其中， y_i 是实际值， $\hat{y}_i$ 是预测值， n 是样本数量。通过最小化 MSE ，模型可以找到使分裂效果最好的特征和阈值。

在本研究中，我们对模型进行了一些创新改进。传统的随机森林采用所有树等权平均的方法，而在本研究中，为了更好地适应高密度 WLAN 网络的特点，采用了加权平均机制。我们在不同的 AP 场景（如 2AP、3AP 场景）中分别训练了多组随机森林模型，并通过加权平均的方法对这些模型的输出进行集成。每个场景下的模型根据 AP 的信道占用和业务流量模式进行细化调整，确保模型在不同场景下的准确性。由于每棵树的权重与其在

训练数据中的表现相关，即基于每棵树的预测误差对其进行权重分配。公式修改为：

$$\hat{y} = \frac{\sum_{i=1}^N \omega_i f_i(x)}{\sum_{i=1}^N \omega_i} \quad (3-10)$$

其中， ω_i 是第*i*棵树的权重， $\hat{y}$ 为加权平均后的最终预测值。这种改进能够更好地利用模型的性能，通过更高权重赋予表现更好的树，提升了整体预测的准确性。

(2) 特征选择优化

在特征选择上，传统的随机森林模型倾向于使用全部特征，可能引入一些无关或冗余特征。在本研究中，通过引入信息增益与相关性分析对特征进行筛选，从而减少无关数据。特征选择的具体方法如下：

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v) \quad (3-11)$$

其中， $Gain(S, A)$ 是特征*A*对数据集*S*的信息增益， $Entropy(S)$ 是数据集*S*的信息熵， S_v 是在特征*A*取值为*v*的子集。通过计算每个特征的信息增益，保留信息增益较高的特征，最终构建更精简和有效的模型。

3.2.5 模型求解

执行以上步骤，使用随机森林模型训练数据集，从而求解出每个特征的影响排行，如图 3-4、图 3-5 所示。全部数值在后续 3.1.4 结果分析中具体分析。

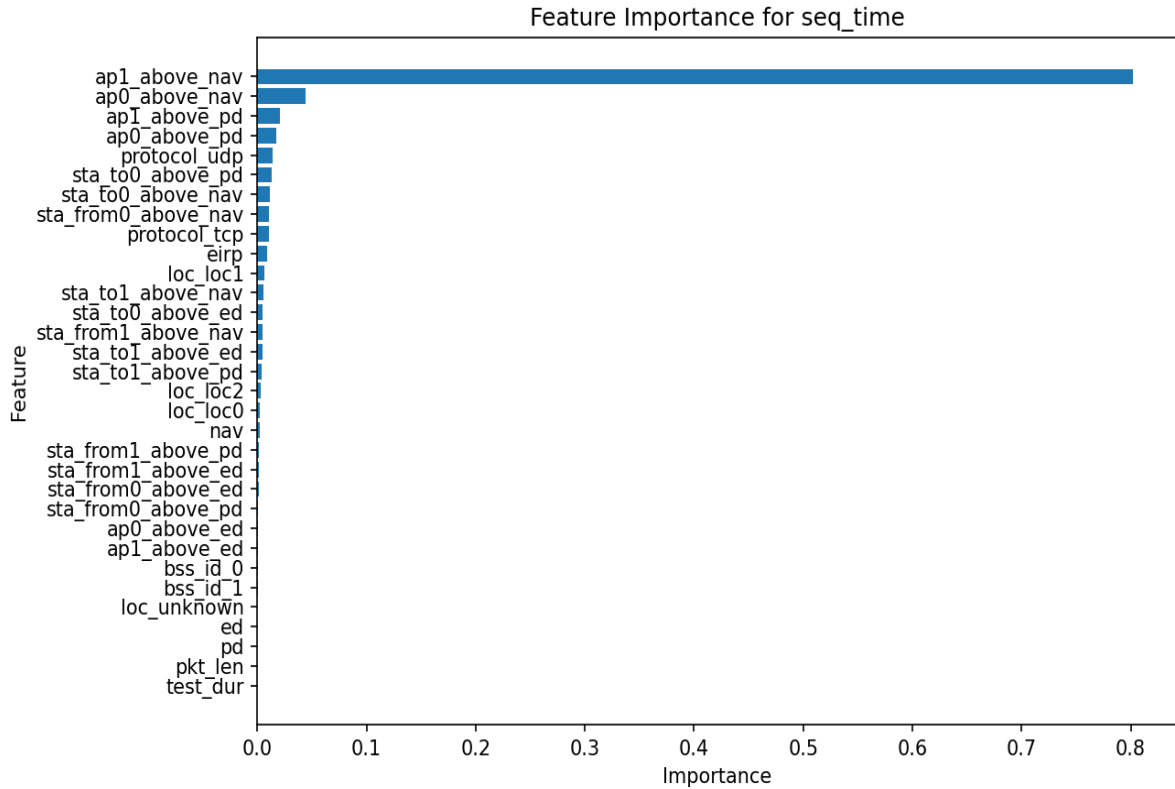


图 3-4 2AP 系统特征的影响排行图

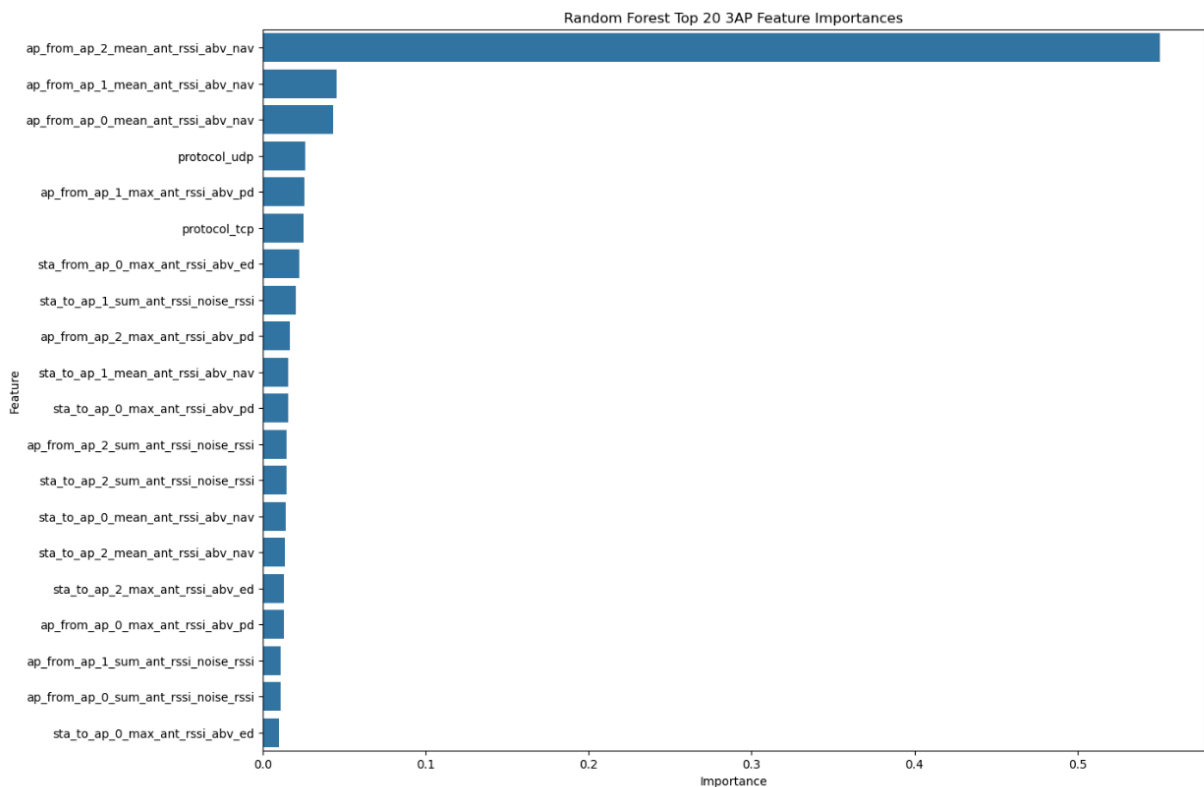


图 3-5 3AP 系统特征的影响排行图

读取测试集，使用训练得到的模型对测试集的样本进行预测，部分结果如表 3-1 所示。

表 3-1 seq_time 预测结果表（部分）

2AP			3AP		
test_id	ap_id	predict seq_time	test_id	ap_id	predict seq_time
1	ap_1	44.570225	1	ap_2	28.09175
1	ap_0	46.35234	1	ap_1	40.07205
2	ap_1	46.196785	1	ap_0	35.99155
2	ap_0	50.560932	2	ap_2	32.1087
3	ap_1	46.009468	2	ap_1	38.5059
3	ap_0	46.584026	2	ap_0	37.2843
4	ap_1	47.644505	3	ap_2	29.6811
4	ap_0	50.801098	3	ap_1	44.6339
5	ap_1	45.901093	3	ap_0	37.77975
5	ap_0	46.559742	4	ap_2	28.7342
...	...	...	...	...	...

3.2.4 结果分析

将 2AP 系统和 3AP 系统训练得到的同类特征进行聚合，得到了 7 个有影响的主要特征，分别将它们的影响排序，如表 3-2 和表 3-3 所示。

表 3-2 2AP 系统主要特征影响排行表

2AP 特征	含义	影响
ap_from_ap	AP 和 AP 间的 RSSI	87.96%
btw_sta_ap	AP 和 STA 间的 RSSI	7.01%
protocol	协议类型	2.32%
loc_id	网络拓扑	1.27%
eirp	信号功率	1.22%
nav	NAV 门限	0.18%
bss_id	BSS 编号	0.04%

表 3-3 3AP 系统主要特征影响排行表

3AP 特征	含义	影响
ap_from_ap	AP 和 AP 间的 RSSI	74.94%
btw_sta_ap	AP 和 STA 间的 RSSI	18.14%
protocol	协议类型	5.12%
loc_id	网络拓扑	0.74%
eirp	信号功率	0.72%
nav	NAV 门限	0.19%
bss_id	BSS 编号	0.16%

因此，训练集中影响 AP 发送机会的参数的影响性强弱排序是（由强到弱）：

AP 和 AP 间的 RSSI >> AP 和 STA 间的 RSSI

> 协议类型 > 网络拓扑 > 信号功率 > NAV 门限值 > BSS 编号

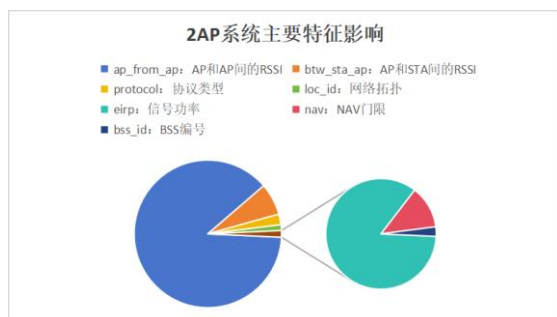


图 3-6 2AP 系统主要特征影响饼状图

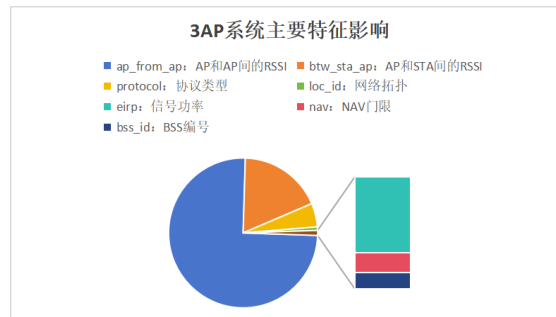


图 3-7 3AP 系统主要特征影响饼状图

四、问题二的建模与求解

4.1 问题分析

除了发送机会外，WLAN 网络的吞吐量还受到 PHY 速率和数据帧大小的影响。AP 在向不同 STA 发送数据时选择的 (MCS, NSS) 组合与信道状况紧密相关。因此，我们需要基于实测数据，构建模型来预测 AP 发送数据时最常使用的 (MCS, NSS) 组合。

问题二的核心是通过分析 SINR (信噪干扰比) 与传输方式之间的关系，预测 AP 在不同情况下使用的 PHY Rate，并根据 PHY Rate 推断出 (MCS, NSS) 的组合类型。我们将根据节点间的 RSSI 信息、PHY Rate、AP 的传输方式和干扰情况进行分析。

(1) 数据分析与处理

通过观察 2AP 数据，可以发现同一实验组中，AP 使用的 (MCS, NSS) 组合会有所不同。因此，我们需要对每条数据进行独立分析。尽管实验中的组合差异不大，但我们可以认为 (MCS, NSS) 组合与实验中多个数据项 (如 RSSI、PHY Rate 等) 相关。我们从多个实验数据中提取 dss (数据流类型) 的信息，结合 RSSI 和 PHY Rate 进行分析。

(2) PHY Rate 与 (MCS, NSS) 组合的映射关系

根据数据分析，可以推断 (MCS, NSS) 组合与 PHY Rate 存在映射关系。我们通过观测和实验数据，构建一个 (MCS, NSS) 组合与 PHY Rate 的映射表格，预测 PHY Rate 的范围，并进一步推断其对应的 (MCS, NSS) 组合。

(3) 计算 SINR

PHY Rate 与 AP 的传输方式密切相关，信号强度 (RSSI) 和信干噪比 (SINR) 是关键因素。通过计算 STA 与不同 AP 之间的 RSSI 差异，我们可以得出 STA 接收数据时的 SINR。SINR (单位为 dB) 的计算方式如下：

$$\begin{cases} SINR = RSSI_{AP-STA} - Noise, & \text{同步传输} \\ SINR = RSSI_{AP-STA} - (RSSI_0 + Noise), & \text{异步传输} \end{cases} \quad (4-1)$$

其中， $RSSI_{AP-STA}$ 是关联 AP 到 STA 的信号强度， $RSSI_0$ 是邻近 AP 对 STA 的干扰强度， $Noise$ 是环境噪声基准值。SINR 的高低将直接影响 PHY Rate 的选择，并通过此信息预测 (MCS, NSS) 组合。

(4) 数据清理与处理

由于实验中存在一些异常数据点，我们将使用去除最大 10% 和最小 10% SINR 值的策略，来确保分析过程的稳健性。这些极端数据可能由于环境噪声或其他外部干扰因素导致，去除后能够更加准确地反映真实的信道状况。

在解决问题二时，我们根据 RSSI 数据计算 SINR，然后通过 PHY Rate 的预测范围，推测最有可能的 (MCS, NSS) 组合。模型的输入包括 AP 与 STA 之间的信号强度，模型输出的 PHY Rate 与 (MCS, NSS) 组合之间的关系则通过预先构建的映射表得到。

然后，采用去除异常值 (最大 10% 和最小 10%) 的方法进行数据清理，进一步优化模型的预测精度。接着，使用问题一中的预测结果来估算同步与异步传输的比例，从而更准确地预测 PHY Rate。

通过这种方式，我们可以确定网络中的主要传输方式，并最终预测 AP 最常用的 (MCS, NSS) 组合，反映了网络的 SINR 水平和 PHY Rate。

4.2 模型构建与求解

我们选择使用卷积神经网络（CNN）作为核心模型进行分析和预测。CNN 擅长处理具有空间依赖性的数据，能够通过卷积操作有效提取输入矩阵中的局部和全局特征。在无线通信网络中，不同 AP 与 STA 之间的信号传输及干扰情况可以通过 RSSI 和 SINR 值表示，这些值构成的矩阵具有空间结构，适合通过 CNN 进行处理和特征提取。为了确保模型能够准确、高效地学习，这之前，我们需要对原始数据进行预处理，以提高模型的收敛速度和预测精度。

4.2.1 数据预处理

对于无线通信系统中的信号传输建模，预处理的主要任务是处理 RSSI 和 SINR 等关键特征数据。

(1) 数据清洗

a) 缺失值处理

在实际数据集中，某些 AP 或 STA 由于物理环境的干扰或设备问题，导致部分 RSSI 或 SINR 数据缺失。我们通过对缺失数据进行分析，对缺失数据进行了填补，使得后续训练过程可以正常进行。

b) 异常值检测

我们使用 Z-score 方法检测了 RSSI 和 SINR 中的异常值，定义 z 分数超过 3 倍标准差的点为异常。检测到的异常值已通过剔除极端值（即去除最大 10% 和最小 10%）的策略进行了清理，从而提高数据的稳健性和模型训练的准确性。Z-score 的计算公式如下：

$$Z = \frac{X - \mu}{\delta} \quad (4-2)$$

其中， Z 值表示当前数据点距离数据集平均值的标准差个数， X 是数据集中某一列的具体数值， μ 表示数据集中某一列的平均值， δ 是数据集的标准差。

(2) Akima 插值拟合算法

在数据处理与分析中，我们发现，不同实验中样本的测量间隔不同，这导致了 RSSI 的测量值数目不同，在特征处理时无法统一计算，因此需要对测量数据的数目进行统一。

经过对数据集的 RSSI 测量值长度进行分析，我们发现：虽然多数测量数据的数量不高于 120，但是在部分实验中普遍超过 400，且测试集中测量数据也普遍较多。为了尽可能不丢失 RSSI 的数据特征，我们选择将所有 RSSI 测量数据统一扩充到较大的数值。

经过分析，最长的 RSSI 测量值队列长度为 521，因此我们选择将数据量统一为 540 个。在保证数据变化趋势不失真的前提下，我们决定采用 Akima 插值拟合算法对原始数据进行补充。

为了计算多项式的斜率，首先需要定义每段之间的差分斜率 m_i ：

$$m_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i} \quad (4-3)$$

m_i 数据点 i 和 $i+1$ 之间的斜率， x_{i+1} 、 x_i 是原始数据点的索引值， y_{i+1} 、 y_i 是原始数据点的数值。

为了平滑地过渡相邻的斜率，Akima 插值通过加权计算每个数据点的最终插值斜率 S_i ：

$$S_i = \frac{|m_{i+1} - m_i|}{|m_{i+1} - m_i| + |m_i - m_{i-1}|} \quad (4-4)$$

其中， S_i 是加权斜率来衡量每个数据点的最终插值斜率， m_{i+1} 、 m_{i-1} 是邻数据点之间

的局部斜率。

利用这些局部斜率，构造出拟合曲线的梯度。这些梯度由 Akima 插值方法进行加权平均，确保每个数据点处的曲线是平滑且不产生过度振荡的。在 Akima 插值中，插值点位于数据点之间的局部曲线上，这些曲线由数据点的前后几组点决定。每一段曲线可以表示为三次多项式：

$$Y_{new} = f(x) = \alpha_3(x - x_i)^3 + \alpha_2(x - x_i)^2 + \alpha_1(x - x_i) + \alpha_0 \quad (4-5)$$

其中， α_i 是多项式系数，这些系数通过相邻数据点之间的斜率来确认， x_i 为插值区间的起点索引。

因此，在整个 Akima 插值中，原始数据点之间的斜率 m_i 通过相邻点的差分计算得到，而拟合曲线的斜率由这些局部斜率通过加权平均计算，从而生成平滑的三次多项式曲线。最终，插值结果 Y_{new} 由这些三次多项式对新索引 X_{new} 得到。

通过 Akima 插值拟合，我们能够将原始数据列补充至固定长度 540，且在处理过程中保持数据的平滑性和趋势一致性。

(3) 特征清洗

经过对问题的分析，我们将 WLAN 组网的网络拓扑简化，如图 4-1 所示（以 2AP 系统为例）。

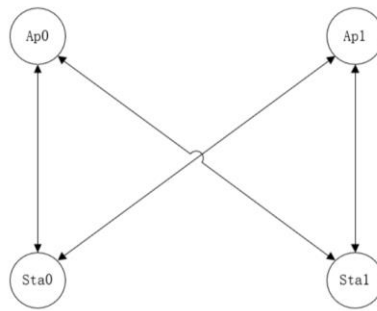


图 4-1 2AP 网络的简化拓扑图

其中，我们将每个节点接收到的信号分为传输信号和噪声信号。例如，在 2AP 系统中，对于 sta_0 来说，sta_from_ap_0_rssi 就是传输信号，而 sta_from_ap_1_rssi 是噪声信号；对于 ap_0 来说，sta_to_ap_0_rssi 是传输信号，sta_1 的 sta_to_ap_0_rssi 是噪声信号。同理，在 3AP 系统中每一个 STA 和 AP 都会接收到 1 个传输信号和 2 个噪声信号。

分别对 RSSI 值的 sum、mean、max 测量值矩阵执行矩阵减法运算，即可得到适用于模型求解的特征：*sta_from_ap_sum_sinr*、*sta_from_ap_mean_sinr*、*sta_from_ap_max_sinr*、*sta_to_ap_sum_sinr*、*sta_to_ap_mean_sinr*、*sta_to_ap_max_sinr*。在 3AP 系统中，由于有 2 噪声信号的存在，所以上特征各有 2 个值。

上述提出的特征提取过程可通过如下的伪代码实现：

算法 4-1: 预处理 STA 和 AP 的 SINR

1. // 计算 STA 的 SINR
 2. **for** *index, row* $\in$ *combined_df* **do**
 3. *sta_id* = *row*[] // 获取当前 STA 的 ID
 4. *other_sta_ids* = [0,1,2] *excluding sta_id* // 获取其他 STA 的 ID
 5. **for** *agg_func* $\in$ ['sum','mean','max'] **do**
 6. *sta_from_ap_current* // 当前 STA 的 RSSI
 7. *sta_from_ap_other_0* // 其他 STA0 的 RSSI
-

```

8.      sta_from_ap_other_1 // 其他 STA1 的 RSSI
9.      sinr_0 // 计算 SINR 0
10.     sinr_1 // 计算 SINR 1
11.     combined_df.at[] = sinr_0 转为列表并存储
12.     combined_df.at[] = sinr_1 转为列表并存储
13.     end for
14. end for
15. // 计算 AP 的 SINR
16. for index,row ∈ combined_df do
17.     ap_id = row[] // 获取当前 AP 的 ID
18.     other_ap_ids = [0,1,2] excluding ap_id // 获取其他 AP 的 ID
19.     for agg_func ∈ ['sum','mean','max'] do
20.         sta_to_ap_current //当前 AP 的 RSSI
21.         sta_to_ap_other_i //获取的是邻接行的 RSSI 值
22.         sinr_0 // 计算 SINR 0
23.         sinr_1 // 计算 SINR 1
24.         combined_df.at[] = sinr_0 //转为列表并存储
25.         combined_df.at[] = sinr_1 //转为列表并存储
26.     end for
27. end for
28. // 处理 SINR 列以去除括号并转换为列表
29. for agg_func ∈ ['sum','mean','max'] do
30.     for i ∈ range(2) do
31.         col_name = 'sta_to_ap_{agg_func}_sinr_{i}'
32.         //存储为字符串的 SINR 列表转换为实际的整数列表
33.     end for
34. end for
35. // 处理 STA 的 SINR 列以去除括号并转换为列表
36. for agg_func ∈ ['sum','mean','max'] do
37.     for i ∈ range(2) do
38.         //存储为字符串的 SINR 列表转换为实际的整数列表
39.     end for
40. end for
41. // 删除不必要的列
42. columns_to_drop_rssi = [] // 找出所有包含 'rssi' 的列
43. combined_df = combined_df.drop() // 删除包含 'rssi' 的列
44. columns_to_drop_additional = [] // 定义其他需要删除的列
45. combined_df = combined_df.drop() // 删除上述定义的多余列

```

(4) 数据集划分

为了评估模型的中间表现，并且避免过拟合和欠拟合，需要对数据集进行划分，划分为训练集和验证集。合理的数据集划分是模型成功的关键步骤之一。在该问题中，我们抽

取了部分训练集数据作为验证集，在模型训练过程中用验证集的预测准确率评估模型的性能与泛化能力，帮助选择最优秀的模型作为最终预测结果的模型。

(5) 特征映射

在对 2AP 系统进行建模时，我们观察数据集发现：当 PHY Rate > 100Mbps 时，NSS=2，而 PHY Rate < 100Mbps 时，NSS=1。由此，可以建立 PHY Rate 对(NSS,MCS)的映射关系。

因此我们在处理训练集数据时先根据对应关系（如表 4-1 所示）将(NSS,MCS)转换为 PHY Rate，再对 PHY Rate 进行建模求解。完成模型训练后，对测试集样本的 PHY Rate 进行预测，并通过映射关系得到(NSS,MCS)的预测值。

表 4-1 (NSS,MCS)和 PHY Rate 的对应关系表

MCS	0	1	2	3	4	5	6	7	8	9	10	11
NSS=1	8.6	17.2	25.8	34.4	51.6	68.8	77.4	86.0	103.2	114.7	129.0	143.4
NSS=2	17.2	34.4	51.6	68.8	103.2	137.6	154.9	172.1	206.5	229.4	258.1	286.8

4.2.2 模型选择和优化

我们分析多个 AP 与 STA 之间的网络拓扑，即每个终端设备从多个 AP 接收信号，这些信号由有用的传输信号和噪声信号组成。通过分析这些信号的干扰情况，我们可以计算每个设备的 SINR（信号干扰噪声比）。随后，我们将这些 SINR 值拼接成矩阵，并输入到卷积神经网络（CNN）中进行进一步的分析和预测。接下来，我们将详细分析该模型的构建过程、公式以及相关的参数。

(1) 计算每个设备的 SINR

对于每个设备 STA_i ，接收的信号可以分为来自目标 AP_m 的有效信号，以及来自邻区 AP_n 的干扰信号和环境噪声。例如，对 2AP 的网络拓扑，设定 STA_1 和 STA_2 分别从 AP_1 和 AP_2 接收信号。对于每个 STA，接收的信号分为目标 AP 的有效信号和邻区 AP 的干扰信号，我们计算每个设备的 SINR 值。

STA_1 从 AP_1 接收有效信号，从 AP_2 接收干扰信号。因此， STA_1 的 SINR 公式为：

$$SINR_{STA_1-AP_1} = RSSI_{AP_1-STA_1} - (RSSI_{AP_2-STA_1} + Noise) \quad (4-6)$$

STA_2 从 AP_2 接收有效信号，从 AP_1 接收干扰信号。因此， STA_2 的 SINR 公式为：

$$SINR_{STA_2-AP_2} = RSSI_{AP_2-STA_2} - (RSSI_{AP_1-STA_2} + Noise) \quad (4-7)$$

推广到 3AP 的网络拓扑中也是类似的计算方式，因此我们设定了统一的 SINR 计算公式：

$$SINR_{STA_i-AP_m} = RSSI_{AP_m-STA_i} - (RSSI_{AP_n-STA_i} + Noise) \quad (4-8)$$

其中， $RSSI_{AP_m-STA_i}$ 表示 STA_i 从 AP_m 接收有效信号，从 AP_n 接收干扰信号， $Noise$ 表示环境噪声基准值。

(2) 数据矩阵的构建

我们通过计算出的 SINR 值，构建一个矩阵，该矩阵可以作为 CNN 的输入。在此模型中，SINR 值可以按照设备 STA 和 AP 的连接关系形成一个二维矩阵。

对于每个设备 STA_i ，接收信号的强度可以表示为一个矩阵。例如，假设我们有以下 SINR 矩阵：

$$\begin{bmatrix} SINR_{STA_i-AP_m} & SINR_{STA_i-AP_n} & SINR_{STA_i-AP_t} \\ SINR_{STA_j-AP_m} & SINR_{STA_j-AP_n} & SINR_{STA_j-AP_t} \\ SINR_{STA_k-AP_m} & SINR_{STA_k-AP_n} & SINR_{STA_k-AP_t} \end{bmatrix} \quad (4-9)$$

该矩阵表示每个 STA_i 从不同 AP_m 接收到的信号强度以及干扰情况。通过这个矩阵，我们可以分析无线网络中各 AP 与 STA 之间的信号传输状况。

(3) CNN 模型的输入与卷积层分析

在构建的模型中，SINR 矩阵作为输入传入到卷积神经网络（CNN）。卷积神经网络擅长从矩阵形式的数据中提取局部和全局特征，尤其适用于分析空间依赖性强的数据。

CNN 的卷积层可以使用不同的卷积核尺寸来提取信号传输的局部特征。对于不同大小的矩阵，我们可以使用自适应池化层（Adaptive Pooling Layer）来调整矩阵的尺寸，使其达到固定大小。这样可以保证网络在处理不同大小的输入时具有灵活性。具体操作上，采用 2D 卷积操作，通过多个卷积核（filters）来提取不同的信号特征。模型设有输入矩阵 X （表示不同设备和 AP 之间的 SINR 值），卷积操作可以表示为：

$$Z = f(W * X + b) \quad (4-10)$$

其中， W 表示卷积核的权重矩阵， b 表示偏置向量， f 表示激活函数—ReLU函数。经过卷积层处理后，得到的输出特征图可以用于分析无线信号传输中的干扰和信道质量。

(4) 自适应池化与特征提取

为了保证 CNN 的输入尺寸一致，我们使用了自适应池化层。自适应池化层可以根据输入矩阵的大小，动态调整池化区域的尺寸，将矩阵缩放到固定的大小。自适应池化的公式为：

$$h' = \left\lfloor \frac{h}{k} \right\rfloor \quad (4-11)$$

$$\omega' = \left\lfloor \frac{\omega}{k} \right\rfloor \quad (4-12)$$

其中， h 和 ω 分别表示为输入矩阵的高度和宽度， k 是池化因子， h' 和 ω' 是池化后的矩阵高度和宽度。池化操作确保可以处理不同大小的SINR矩阵输入，同时保留了重要的空间特征。

(5) 参数分析

在问题二的模型中，关键的参数包括：

a) SINR计算中的信号强度和干扰强度

这些数据可以通过实际测量的RSSI值来确定。模型中的参数调整主要集中于如何精确地建模信号强度和干扰强度的影响。

b) 卷积核大小

卷积核的大小决定了 CNN 捕捉局部特征的能力。较大的卷积核可以捕捉全局特征，而较小的卷积核则适合捕捉局部变化。

c) 池化层设置

池化层的设置可以影响模型处理不同尺寸输入的能力。自适应池化使得我们可以处理不同维度的输入，同时保持输出尺寸一致性。

(6) CNN 模型优化

a) 残差连接

为了优化 CNN 的深层结构，引入残差网络的思想，加入残差连接。在网络中增加残差连接可以防止梯度消失，同时能够更好地传递低层次的信息。

残差连接的公式为：

$$y = F(x, \{W_i\}) + x \quad (4-13)$$

其中， $F(x, \{W_i\})$ 是表示输入 x 经过卷积、激活函数等非线性变换后的结果， $\{W_i\}$ 是卷积层的权重。

b) 均方误差

在输出层，CNN 模型的任务是预测最常用的（MCS, NSS）组合。我们采用均方误差作为损失函数。 MSE 用于衡量模型预测值和真实值之间的差异，并通过最小化误差来优化模型的预测性能， MSE 公式：

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (4-14)$$

其中， L 是损失值， N 是数据集中的样本数， y_i 是第 i 个样本的真实值， $\hat{y}_i$ 是第 i 个样本的预测值，模型通过不断调整参数，最小化预测值和真实值之间的平方误差。

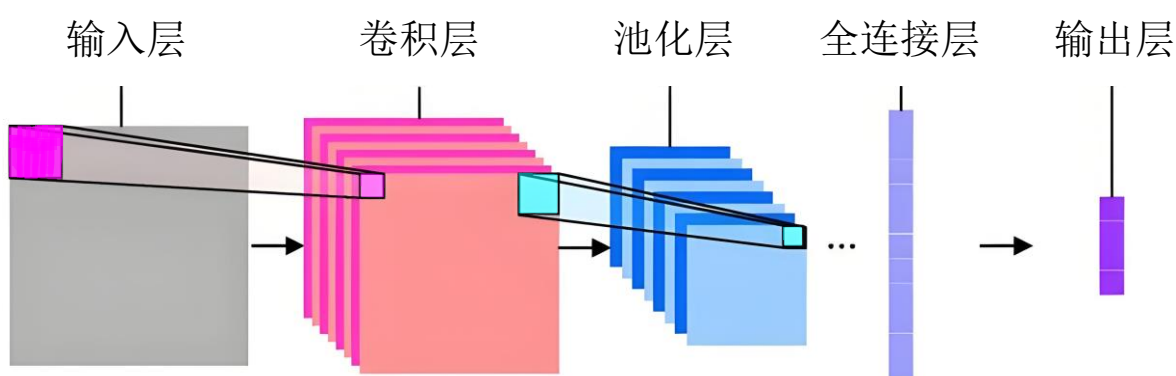


图 4-2 CNN 模型架构

4.2.3 模型求解

执行上述步骤，使用卷积神经网络对特征进行训练。

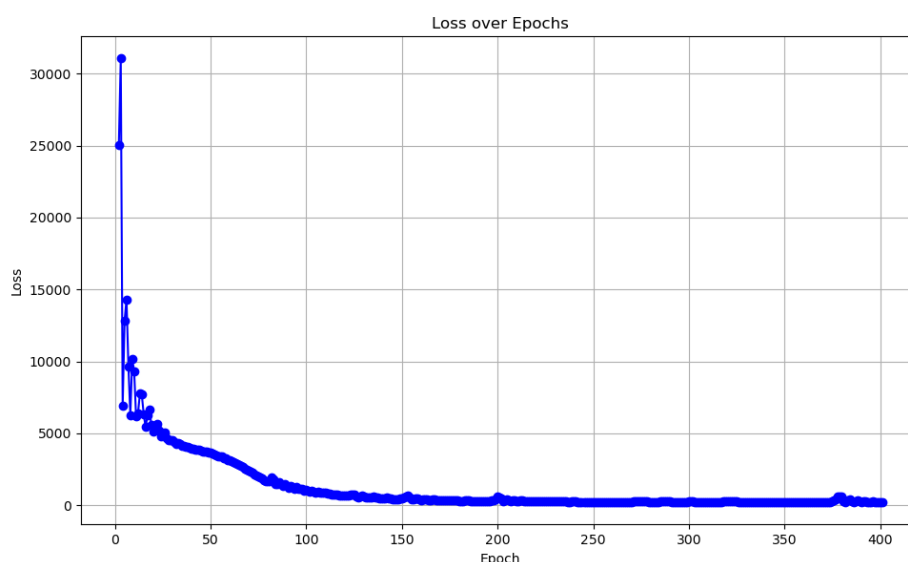


图 4-3 2AP 系统的(MCS,NSS)预测模型 loss-epoch 图

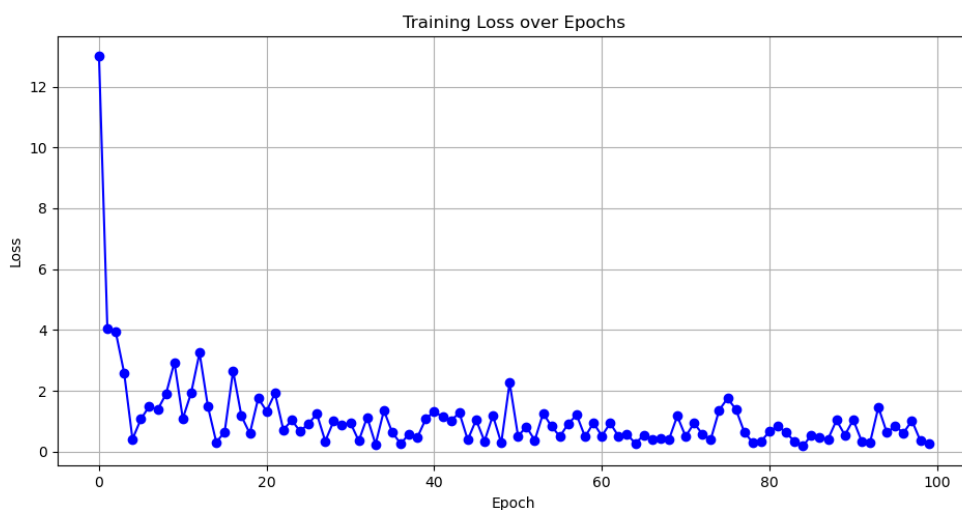


图 4-4 3AP 系统的(MCS,NSS)预测模型 loss-epoch 图

随着训练的 epoch 数增加，模型的 loss 数值不断降低，最终根据在验证集上的预测准确率我们选择了最合适的模型进行测试集数据的(MCS,NSS)预测，部分结果如表 4-2 所示。

表 4-2 (MCS,NSS)预测结果表（部分）

2AP				3AP			
<i>test_id</i>	<i>ap_id</i>	<i>predict nss</i>	<i>predict mcs</i>	<i>test_id</i>	<i>ap_id</i>	<i>predict nss</i>	<i>predict mcs</i>
1	ap_1	2	8	1	ap_2	2	5
1	ap_0	2	8	1	ap_0	2	11
2	ap_1	2	8	1	ap_1	2	10
2	ap_0	2	8	2	ap_2	2	5
3	ap_1	2	8	2	ap_0	2	11
3	ap_0	2	7	2	ap_1	2	10
4	ap_1	2	8	3	ap_2	2	5
4	ap_0	2	7	3	ap_0	2	11
5	ap_1	2	7	3	ap_1	2	9
5	ap_0	2	7	4	ap_2	2	5
...	...	...	...	...	...	...	...

4.2.4 结果分析

在进行 2AP 系统的模型构建时，我们先对(MCS,NSS)的组合进行了编码，将问题直接转换为了 25 分类问题。

然而，在对该问题进行建模求解的过程中，我们发现它的 loss 值维持在一个数值不再变化，且在验证集上的准确率也保持不变，如图 4-5 所示。

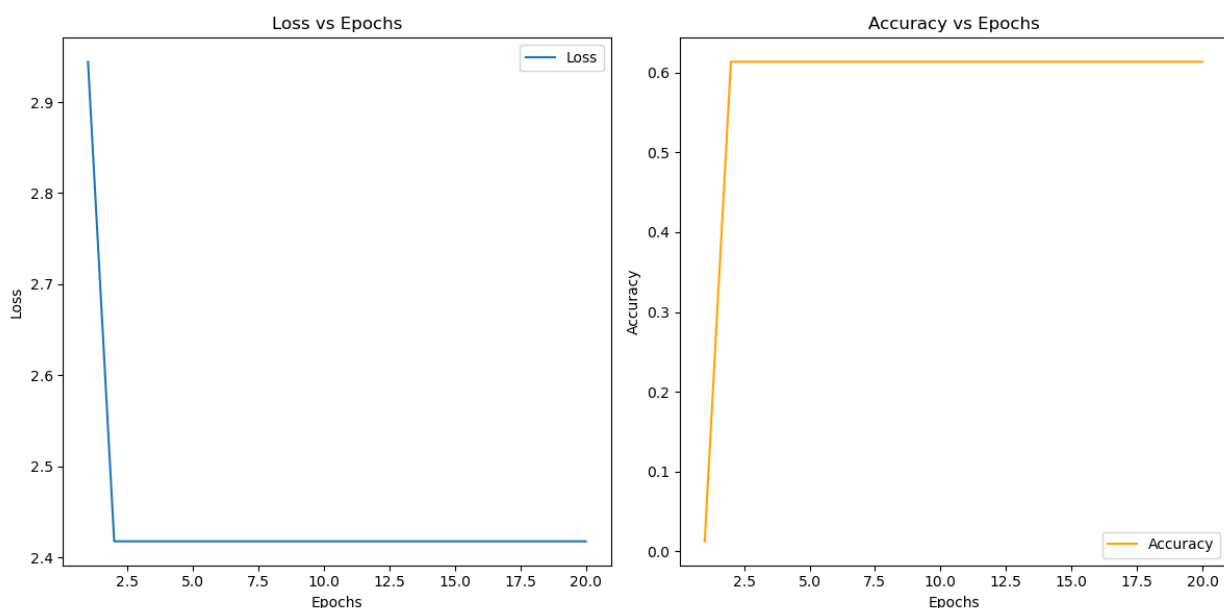


图 4-5 2AP 系统直接对(MCS,NSS)进行预测的 loss 和准确率变化图

观察预测结果发现,这样的模型对所有样本的预测均为第 15 类,即 NSS=2, MCS=11。因此,这种模型无法使用。我们选择重新观察数据中 PHY Rate 与(MCS, NSS)的映射关系,然后对 PHY Rate 进行建模,达成了在验证集上预测的更高准确率。

最终选用的模型在预测(MCS,NSS)的验证集上取得了 86.73%的准确率,充分说明了这一模型具有足够的准确度和泛化能力,能够有效地运用在对实际问题的预测中。同时,观察表 4-2 中的预测结果可以发现,多数情况下 NSS 的数值为 2,这与我们先前对数据集的观察是一致的。

五、问题三的建模与求解

5.1 问题分析

在 WLAN 网络中,系统吞吐量不仅依赖于发送机会,还受到物理层速率(PHY Rate)和数据帧大小的影响。AP 通过自适应调制编码(AMC)算法选择调制编码方案(MCS, NSS),其选择受传输方式、干扰情况、信道条件以及 RSSI 的影响。我们需要根据问题一和问题二中的分析,构建预测模型,利用实测数据中的真实(MCS, NSS)组合来预测网络吞吐量。

问题三的核心在于系统吞吐量的建模不仅要考虑多个 AP 之间的竞争情况、业务流量的类型(如 TCP 和 UDP),还要将二进制指数退避算法、竞争窗口(CW)、网络分配矢量(NAV)等因素纳入模型。吞吐量的变化与信道状况密切相关,因此我们需要根据不同同频 AP 的数量分类建模,或者统一建模。

(1) 自适应调制编码(AMC)

MCS (Modulation and Coding Scheme) 和 NSS (Number of Spatial Streams):这两个参数决定了 PHY 速率。高 MCS 和 NSS 能够支持更高的吞吐量,但对信道质量要求更高

(2) 二进制指数退避与竞争窗口的影响

无线网络中的每个 AP 通过二进制指数退避算法竞争信道,每次冲突后竞争窗口(CW)

增大，增加了传输的延迟。因此，吞吐量不仅取决于 PHY Rate，还与 AP 在竞争过程中等待信道空闲的时间有关。在传输失败后，AP 会在 ACKTimeout 后重传，竞争窗口翻倍，CW 的大小直接影响数据包传输的频率。需要在模型中考虑 CW、DIFS（DCF 帧间距）以及退避计数器的变化对吞吐量的影响。

(3) 业务流量类型的影响

业务流量的类型也是影响吞吐量的重要因素。对于 UDP 流量，报文的大小和发送时间间隔影响吞吐量；对于 TCP 流量，发送方需要根据网络拥塞情况调整发送速率。当丢包率高时，TCP 流量的速率下降，反之，当网络状况良好时，TCP 流量速率提升。此外，TCP ACK 也是一种数据流，上行与下行流量的不对称性会影响吞吐量的预测。因此，模型中需要分别考虑 TCP 和 UDP 流量的特性，尤其是 TCP 流量中丢包率对吞吐量的动态调节作用。

(4) 聚合机制与吞吐量

在 WLAN 网络中，AMSDU 和 AMPDU 聚合机制允许 AP 将多个数据包聚合成一个帧进行传输，从而提高吞吐量。当小包的数量增加时，聚合机制有效减少了帧头的开销，提高了传输效率。因此，吞吐量的预测不仅依赖于单个数据包的传输速率，还与聚合的 MPDU 和 MSDU 数量有关。我们需要在模型中考虑聚合后的帧长度及其对系统吞吐量的贡献。

对于问题三的求解，我们基于实测数据中的（MCS, NSS）组合，预测每个 AP 的 PHY Rate。然后，通过二进制指数退避算法，结合竞争窗口和信道空闲时间，估算 AP 的传输延迟。我们还根据业务流量类型，考虑 TCP 丢包率对传输速率的调节作用。最后，通过 RTS/CTS 机制与聚合机制，进一步调整模型输出的吞吐量。

5.2 模型构建与求解

为了准确预测系统的吞吐量，我们选择使用卷积神经网络（CNN）作为核心预测模型。CNN 在处理空间依赖性较强的数据时表现出色，尤其适用于提取无线信道中的局部和全局特征。通过将 RSSI、SINR、竞争窗口（CW）、业务流量类型、MCS、NSS 等特征因素作为输入数据，结合问题一的 *seq_time* 和问题二实际测量的（MCS, NSS）组合，我们构建了基于 CNN 的模型求解问题三。

5.2.1 数据预处理

我们首先对实测数据进行了全面的数据预处理，确保输入数据质量和一致性。数据预处理的主要步骤包括：

(1) 缺失值处理

对于部分缺失的 RSSI 和 SINR 数据，我们使用平均值进行填充。

(2) 归一化处理

为了防止不同数据量级影响模型训练效果，我们对所有特征进行了归一化处理，将数据缩放至 [0,1] 范围。

(3) 随机回退处理

二进制指数退避算法是无线网络中常用的机制，控制数据帧发送时的竞争窗口。该算法会直接影响吞吐量的变化，因此在模型中需要考虑每次数据传输时的回退机制。

CW_{min} 和 CW_{max} 在不同节点的竞争窗口初始值可能不同，模型应结合 CW_{min} 和 CW_{max} 来建模数据传输的成功率。

重传 r 次失败后，数据帧会被丢弃，这将直接影响系统的吞吐量。因此，应将重传次数

作为一个关键特征，并且要结合 CW 值对其进行处理。

在数据预处理时，需提取出各节点的 CW_{min} 、 CW_{max} ，重传次数 r ，并根据传输失败次数的累积，来调整发送成功的概率。

(4) 聚合与分包机制

MSDU 和 MPDU 的聚合机制会直接影响吞吐量的计算，特别是对于小包传输来说，聚合可以显著减少信道占用时间，提高有效数据的传输效率。

在数据预处理时，应处理以下关键信息：

AMSDU 和 AMPDU 聚合个数——通过分析每个数据包的聚合情况（MSDU 和 MPDU），可以计算出其对吞吐量的影响。实际数据中的 num_ampdu 和 num_amsdu 这两个特征可以用于表示数据帧的聚合情况。

数据包大小——通过统计不同流量类型下的数据包大小，结合其聚合效率，预测总的吞吐量。

(5) 数据传输类型及丢包处理

数据传输类型（如 TCP 和 UDP）对吞吐量有显著影响。UDP 是单向流，通常用于不要求确认的快速数据传输，而 TCP 是双向流，具有丢包重传机制。丢包率 per 是影响 TCP 吞吐量的关键指标。在预处理阶段：

确认 per ，因为在模型中丢包率的高低对吞吐量影响较大，特别是对 TCP 流量，丢包率较高时，数据重传会降低总体吞吐量。

模型应区分 TCP 和 UDP 流量，且需要计算 TCP 的 ACK 数量与实际数据量的比例，这有助于更准确地预测下行和上行之间的带宽占用情况。针对不同类型的流量制定不同的特征处理方案。TCP 包含重传机制，因此需结合丢包率 per 对其进行处理。

(6) RTS-CTS 机制

通过 RTS 和 CTS 的握手机制，可以有效减少无线网络中数据包的碰撞。但这种机制会额外消耗信道资源，因此，在吞吐量预测模型中，RTS-CT 的开销也是一个重要因素。

数据帧在 RTS 和 CTS 的握手过程中会占用部分信道资源，影响总的吞吐量。因此，在预处理时应提取这些控制帧的开销数据，并将其作为特征之一。如果某个 AP 的碰撞次数较多，则表明其信道竞争激烈，这时 RTS-CTS 机制的开销将会增大，影响吞吐量。

(7) 特征提取

除了 RSSI 和 SINR，也将问题二中的 MCS、NSS 等特征一并纳入模型输入。设定网络中有 n 个 STA，每一行代表一个 STA 接收到的同一个 AP 的不同信号特征，列代表同一 AP 给不同 STA 发出的信号。构建的输入矩阵为：

$$Input_{AP_m} = \begin{bmatrix} RSSI_{STA_1-AP_m} & SINR_{STA_1-AP_m} & \cdots & MCS_{STA_1-AP_m} & NSS_{STA_1-AP_m} \\ RSSI_{STA_2-AP_m} & SINR_{STA_2-AP_m} & \cdots & MCS_{STA_2-AP_m} & NSS_{STA_2-AP_m} \\ \vdots & \vdots & & \ddots & \\ RSSI_{STA_n-AP_m} & SINR_{STA_n-AP_m} & \cdots & MCS_{STA_n-AP_m} & NSS_{STA_n-AP_m} \end{bmatrix} \quad (5-1)$$

5.2.2 网络吞吐量预测模型

在 WLAN 系统中，吞吐量（Throughput）是指在单位时间内成功传输的数据量。假设每个 AP 的发送时长为 $T = seq_time + other_air_time$ ，每个数据帧的大小为 L ，并且丢包率为 per ，则吞吐量公式为：

$$Throughput = \frac{(1 - per) \times L \times num_{ppdu}}{seqtime} \quad (5-2)$$

我们将上述输入矩阵作为 CNN 的输入。CNN 的主要优势在于其卷积层能够通过滤波

器捕捉信号的局部变化，同时池化层帮助减少数据维度，保留关键信息。

(1) 卷积层

基于时间序列的数据，使用一维卷积进行特征提取，通过局部滤波器提取时间步之间的模式。卷积核大小决定了提取模式的范围，使用 3×1 的卷积核可以识别相邻三个时间步之间的关系。卷积层通过滑动窗口的方式处理特征，能够有效识别信号强度、丢包率等特征随时间变化的规律。

(2) 池化层

池化层用于下采样，减少特征维度的同时保留重要信息。使用最大池化，通过提取局部特征中的最大值来代表该区域的关键信息。这有助于减少计算量，同时防止过拟合。

(3) 全连接层

在卷积层提取到足够的局部特征后，全连接层用于整合所有特征并进行最终的预测。全连接层通过多个神经元组合先前卷积层的输出，逐渐将特征映射到吞吐量预测值上。

(4) Adam 优化器优化模型

构建好 CNN 模型后，使用无线网络数据对模型进行训练，选择适当的损失函数和优化器。Adam 优化器结合了动量法和 RMSProp 的优点，使用了一阶矩估计和二阶矩估计来动态调整学习率。

计算损失函数相对于模型参数 θ 的梯度 g_t 、一阶矩估计 m_t （梯度的指数加权移动平均）、二阶矩估计 v_t （梯度平方的指数加权移动平均）：

$$g_t = \nabla_{\theta_t} J(\theta_t) \quad (5-3)$$

$$\begin{cases} m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \end{cases} \quad (5-4)$$

其中， $J(\theta_t)$ 是损失函数， θ_t 是当前时刻的 t 模型参数， β_1 是动量超参数（设置为 0.9）， β_2 是平滑参数（设置为 0.999）。

通过 Adam 优化器动态调整学习率，模型能够更快速且稳定地收敛，最终得到一个能够预测网络吞吐量的 CNN 优化模型。

5.2.3 模型求解

根据上述步骤，使用一维卷积神经网络对网络吞吐量模型进行求解，训练过程中的 loss 如图 5-1、图 5-2 所示。

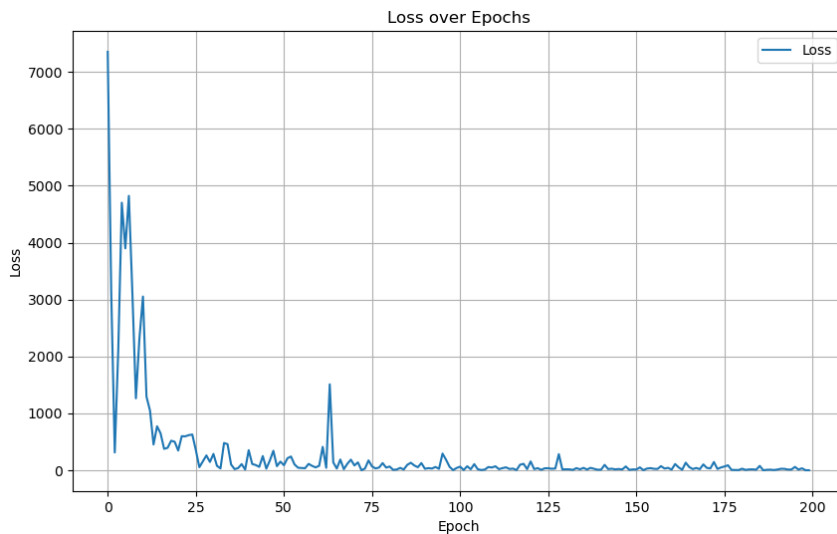


图 5-1 2AP 系统吞吐量预测模型 loss-epoch 图

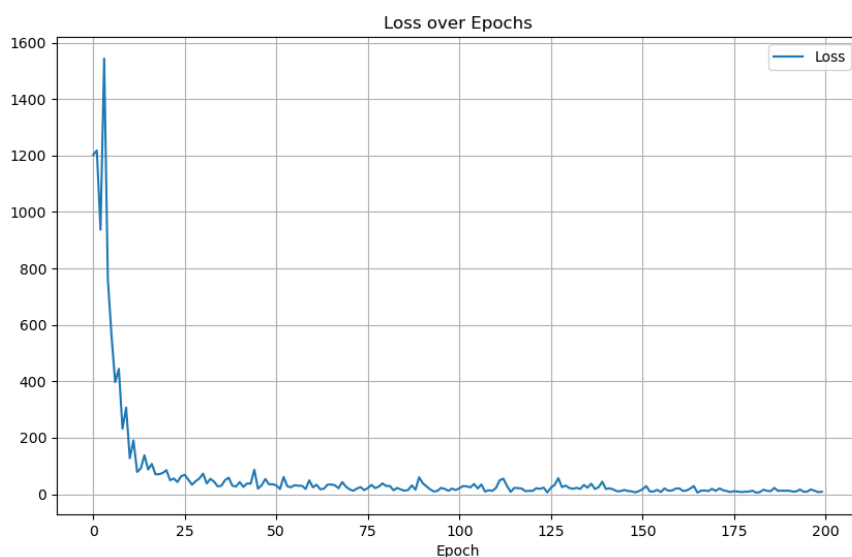


图 5-2 3AP 系统吞吐量预测模型 loss-epoch 图

Loss 值收敛后，选取验证集预测准确度较高的模型作为最终模型，对测试集中的数据进行预测，得到的部分结果如表 5-1 所示。

表 5-1 网络吞吐量预测结果表（部分）

2AP			3AP		
<i>test_id</i>	<i>ap_id</i>	<i>predict throughput</i>	<i>test_id</i>	<i>ap_id</i>	<i>predict throughput</i>
1	ap_1	83.78134	1	ap_2	26.91417
1	ap_0	97.59573	1	ap_1	46.02786
2	ap_1	102.1118	1	ap_0	48.96458
2	ap_0	116.0928	2	ap_2	28.27184
3	ap_1	96.82376	2	ap_1	41.54965
3	ap_0	97.61886	2	ap_0	40.28491
4	ap_1	101.9209	3	ap_2	19.28996
4	ap_0	117.7275	3	ap_1	44.18107
5	ap_1	95.49149	3	ap_0	40.87954
5	ap_0	99.47478	4	ap_2	38.93467
...	...	...	...	...	...

5.2.4 结果分析

预测网络吞吐量的模型在 2AP 系统的验证集中获得了 95%以上的准确率，其中，所有数据预测的平均误差处于 5%左右，在前 25epoch 中，模型处于学习阶段，准确率稳步上升，平均误差总体同样呈下降趋势，说明整个模型在逐步学习并具备预测系统吞吐量的能力，在 epoch 超过 50 后模型性能逐渐趋向稳定；在 3AP 系统的验证集中获得了 95%以上的准确率，模型同样在 epoch 超过 50 后模型性能逐渐趋向稳定。这充分体现了模型的高超性能以及充足的泛化能力。同时，观察表 5.1 中的预测结果可以发现，网络吞吐量符合实际，并且与数据集中的数据相吻合。

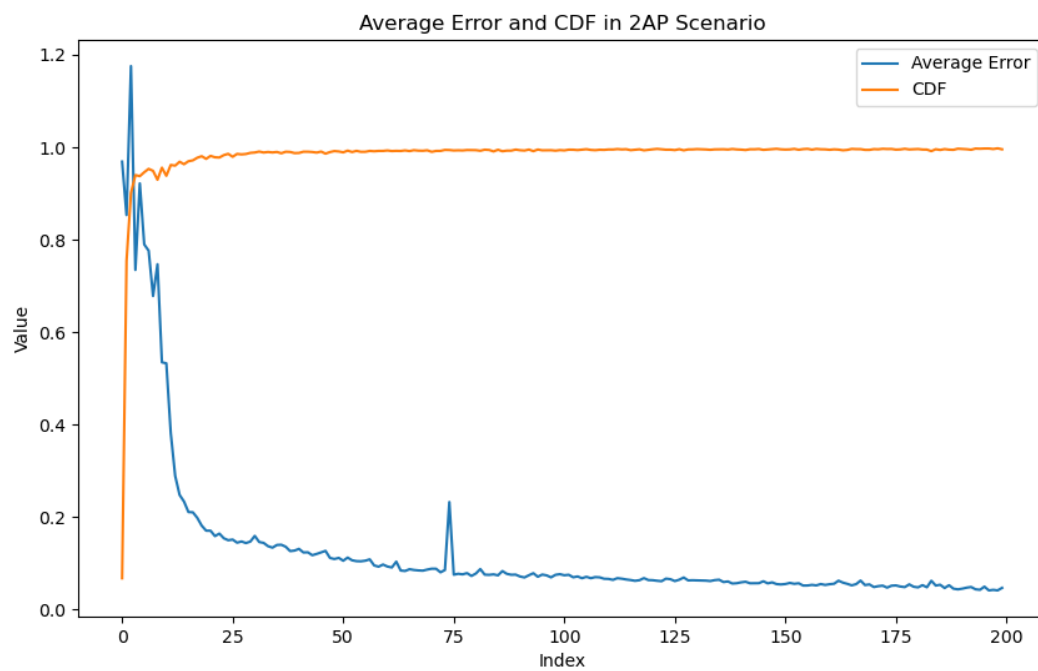


图 5-3 2AP 系统吞吐量预测 CDF 曲线

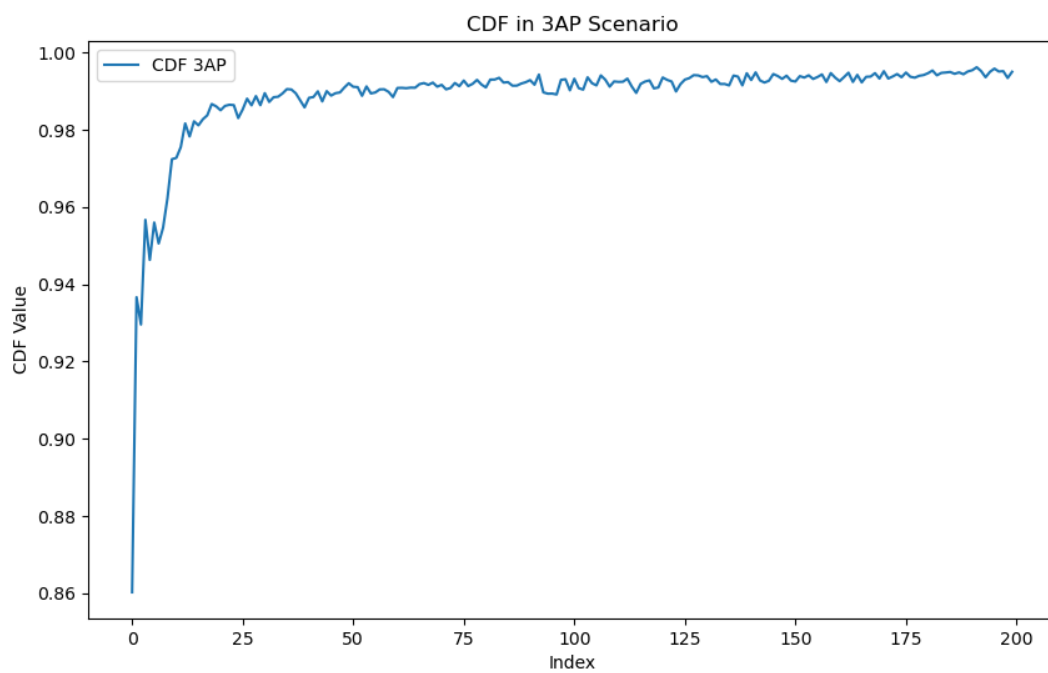


图 5-4 3AP 系统吞吐量预测 CDF 曲线

六、模型的改进与结论

6.1 模型改进建议

在问题三的无线网络吞吐量预测中，卷积神经网络（CNN）虽然擅长处理局部特征和时间序列中的短期模式，但它也有一些不足，尤其是在处理无线网络中的长时间依赖性、非线性复杂性等方面。

处理长时间依赖的能力有限：CNN 擅长捕捉局部的时间序列模式，但对于长期依赖特征的捕捉能力较弱。无线网络吞吐量通常不仅受当前时刻的信号强度和丢包率影响，还与长时间范围内的网络状态有关，而 CNN 可能无法很好地处理这种时间序列的长程依赖性。

缺少对序列特征的捕捉能力：CNN 更多地关注局部信息的卷积提取，但吞吐量预测中涉及的许多特征，比如信道竞争和干扰等，具有长期时间序列依赖。仅使用 CNN 可能会忽略这些时序相关性，无法捕捉较复杂的模式。

为了提高 CNN 模型的性能，针对上述不足，可以进行以下几方面的改进：

(1) 添加 Dropout 层

Dropout 是一种防止过拟合的技术，尤其在面对小规模数据集或复杂的无线网络环境时，Dropout 可以通过随机屏蔽一部分神经元来减少模型对某些特征的过度依赖。其核心思想是每一轮训练中随机忽略部分神经元，使得模型不会过度依赖某些局部特征。这样可以使模型更具泛化能力，从而在测试集上的表现更加稳定。

Dropout 的正则化效果体现在训练过程中，某些神经元被随机忽略（即它们的激活值被设置为零），从而迫使网络学会更加鲁棒的特征表示，减少对训练集的过拟合。可以在 CNN 的卷积层或全连接层后加入 Dropout 层，典型的 Dropout 比率通常在 0.2 到 0.5 之间。

(2) 结合 RNN

CNN 擅长处理局部的时间序列特征，但对于长时间的依赖特征处理能力较弱。在问题三中，吞吐量预测可能涉及到多个时间步的数据，比如信道状态、历史吞吐量、网络负载等。这时，结合 RNN（如 LSTM 或 GRU）可以更好地捕捉这些长时依赖的特征。

RNN 尤其是 LSTM（长短期记忆网络）和 GRU（门控循环单元）是处理时间序列数据的经典模型，它们可以通过记忆单元来捕捉长时依赖特征。LSTM 可以记住长时间范围内的相关信息，特别适合处理有时间延迟的网络变化对吞吐量的影响。

(3) 加入注意力机制

在很多深度学习任务中，注意力机制已经证明能够有效提升模型的性能。注意力机制通过让模型学会关注输入数据中最重要的部分，帮助模型选择对吞吐量预测最相关的特征。通过引入注意力机制，模型可以更加灵活地聚焦于影响网络性能的关键特征，如某些时间段的网络拥塞或信号强度的变化。

6.2 主要结论

本研究通过构建基于卷积神经网络（CNN）的模型，对无线局域网（WLAN）中的吞吐量进行了深入的预测和分析，得出以下主要结论：

(1) 吞吐量与信道条件、流量类型及丢包率等密切相关

吞吐量受到多个因素的综合影响，主要包括：

- a) 信号强度越高，数据传输的可靠性越好，吞吐量越高。RSSI 是衡量信道质量的关

键因素，强信号能够减少数据传输中的丢包率，进而提高吞吐量。

- b) 丢包率越高，意味着网络中传输数据包的错误率增加，导致重传次数增多，从而降低有效吞吐量。减少丢包率是提高吞吐量的关键。
- c) TCP 流量由于重传和确认机制，其吞吐量容易受到丢包率的影响。而 UDP 流量没有重传机制，传输速率较快，但对信道质量要求较高。
- d) 数据包的聚合机制在高负载环境下显著提高了吞吐量。通过将多个数据包聚合成一个更大的数据包传输，可以减少每次传输的信道竞争和控制开销，进而提升整体吞吐量。
- e) MCS 和 NSS 的组合共同决定了无线传输速率，进而影响了吞吐量。在实际应用中，不同的 MCS 和 NSS 组合会产生不同的物理层数据速率（PHY Rate）。

(2) 提高信道中的吞吐量的有效策略

在复杂的无线网络环境中，提升吞吐量可以通过以下策略：

通过减少冲突、提高信道利用率，可以显著提升吞吐量。引入 RTS-CTS 机制可以有效减少信道争用时的数据包碰撞，特别是在高密度网络中。

适当调整清除信道评估（CCA）和网络分配向量（NAV）门限，使 AP 能够在干扰较少的情况下更灵活地占用信道，增加信道的利用效率。

通过提升网络中信号接收的稳定性、增强抗干扰能力，可以减少数据包的丢失和重传，从而提升吞吐量。

使用更有效的 AMSDU 和 AMPDU 数据包聚合机制可以提高信道利用效率，减少信道空闲时间，提高数据的传输速率，最终提升整体吞吐量。

(3) 使用模型预测吞吐量的有效性

本研究中提出的基于 CNN 的模型能够准确预测无线网络中的吞吐量，并为网络管理者提供有价值的参考：

局部特征提取：通过卷积层，模型能够提取影响吞吐量的局部模式和特征，尤其是信号强度、丢包率等局部变化信息，有效预测不同网络条件下的吞吐量变化。

时间序列特性分析：通过对多时间步的信道状态数据进行处理，模型能够预测未来时间段内的吞吐量，从而为网络运营者提供动态优化的依据。

6.3 研究贡献

本研究为无线网络中的吞吐量预测提供了一种新的解决方案，并在深度学习应用于无线网络性能预测方面做出了有益探索。通过综合应用多种模型改进技术，研究有效提升了模型的预测精度和适应能力，为无线网络性能优化提供了新的工具和参考。

(1) 构建基于 CNN 的无线网络吞吐量预测模型

本研究首次将卷积神经网络（CNN）应用于无线局域网（WLAN）的吞吐量预测问题中。通过对无线信号强度（RSSI）、丢包率（PER）、流量类型、聚合机制等多个网络特征的综合分析，建立了一个高效的深度学习模型。该模型利用 CNN 擅长的局部特征提取能力，能够在复杂的无线网络环境下有效预测 AP 的吞吐量。这种方法为传统的吞吐量预测模型提供了新的视角，极大地提高了预测精度。

(2) 拓展无线网络数据处理框架

研究中不仅提出了一种新的基于深度学习的吞吐量预测模型，还对无线网络数据的处理流程进行了系统化设计。包括数据清洗、特征提取与工程、时间序列化等预处理步骤，为后续的网络性能优化研究提供了一个完整的框架。这一框架不仅适用于本研究的问题，还可为未来无线通信中其他预测任务（如信道状态预测、网络延迟预测等）提供参考。

(3) 对无线网络优化的实际应用

本研究提出的吞吐量预测模型具有实际应用价值，特别是在高密度无线网络的部署和优化中。通过准确预测不同接入点（AP）的吞吐量，网络运营商可以更好地管理网络资源，减少信道拥塞，提高网络整体性能。本模型可用于智能网络管理平台，支持动态网络资源调度、负载均衡等优化措施。

(4) 为后续研究提供基线

本研究提出的基于 CNN 和 RNN 的吞吐量预测模型为未来的研究奠定了基线，后续研究可以在此基础上进一步优化网络架构、改进训练算法或引入新的正则化技术，提升模型性能。同时，研究中探索的多特征融合方法和模型架构可以被广泛应用于其他无线网络性能预测任务中，推动该领域的进一步发展。

参考文献

- [1] 朱瑞强. 基于 RSSI 的无线传感网攻击检测与安全路由研究[D]. 南昌大学, 2024. DOI:10.27232/d.cnki.gnchu.2024.001112.
- [2] 甄雪, 张闪闪. 一种分布式的基于预留的多信道 MAC 协议[J]. 计算机技术与发展, 2020, 30(1):54-59. DOI:10.3969/j.issn.1673-629X.2020.01.010
- [3] 孙元凯. 超高速无线局域网链路自适应算法的仿真与实现[D]. 江苏:东南大学,2011. DOI:10.7666/d.Y2054646.
- [4] 谭凯.基于 IEEE 802.11ac 的自适应 MIMO-OFDM 系统的研究[D].广东工业大学,2017.
- [5] 宋海声,朱长驹,吴佳欣,等.基于 RSS 阈值模型的 Amorphous 算法定位误差抑制[J].计算机工程与科学,2018,40(05):821-828.
- [6] 朱加伟.无线局域网安全性测试和视频传输关键技术研究[D].中国科学技术大学,2012.
- [7] 胡国庆.下一代无线局域网中信道接入技术研究[D].西安电子科技大学,2022.DOI:10.27389/d.cnki.gxadu.2022.001846.
- [8] TEGEGNE LEMLEM KASSA. WLAN 中面向 MU--MIMO 的自适应帧聚合方法的最佳吞吐量研究[D]. 四川:电子科技大学,2023.
- [9] 曹珂峻,张闽,张天舒. Wi-Fi 7 关键技术研究综述[J]. 电脑知识与技术,2023,19(29):82-84.
- [10]徐梦颖. IEEE 802.11ax 网络高吞吐量帧聚合方案及其优化[D]. 浙江:浙江工业大学,2023.
- [11]杜浩源. 下一代 WLAN 多 AP 协作技术研究[D]. 陕西:西安电子科技大学,2022.
- [12]刘彤,潘梁,韩锐. WLAN 中一种基于 AP 选择的负载均衡机制[J]. 网络新媒体技术,2021,10(4):25-31. DOI:10.3969/j.issn.2095-347X.2021.04.004.
- [13]黎露. IEEE 802.11ax 中基于强化学习的 MAC 算法研究[D]. 重庆:重庆邮电大学,2021.
- [14]S. Biswas et al., "FairIN: Throughput Fairness in Infrastructure-Based Wireless Access Networks," 2020 International Conference on System, Computation, Automation and Networking (ICSCAN), Pondicherry, India, 2020, pp. 1-6, doi: 10.1109/ICSCAN49426.2020.9262313.
- [15]N. D. Tangudu, S. Kumar, S. K. Vankayala and S. Yoon, "Neural Network Approach to Channel estimation in Wireless Communication," 2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), Bangalore, India, 2024, pp. 1-6, doi: 10.1109/CONECCT62155.2024.10677094.
- [16]B. Bellalta, "Throughput Analysis in High Density WLANs," in IEEE Communications Letters, vol. 21, no. 3, pp. 592-595, March 2017, doi: 10.1109/LCOMM.2016.2632109.
- [17]K. Ermolov, "Throughput modeling in IEEE 802.11 WLAN-based wireless networks," 14th Conference of Open Innovation Association FRUCT, Espoo, Finland, 2013, pp. 31-36, doi: 10.1109/FRUCT.2013.6737942.

附录

附录 1 test_set_1_2ap 结果表

2AP			
test_id	ap_id	predict seq_time	predict throughput
1	ap_1	44.570225	83.78134
1	ap_0	46.35234	97.595726
2	ap_1	46.196785	102.11179
2	ap_0	50.560932	116.09284
3	ap_1	46.009468	96.82376
3	ap_0	46.584026	97.61886
4	ap_1	47.644505	101.920876
4	ap_0	50.801098	117.72748
5	ap_1	45.901093	95.491486
5	ap_0	46.559742	99.47478
6	ap_1	47.546806	101.955734
6	ap_0	50.787483	121.89505
7	ap_1	46.2742	100.282394
7	ap_0	47.634365	117.87666
8	ap_1	47.91772	105.64012
8	ap_0	51.83037	126.324974
9	ap_1	44.172344	109.56798
9	ap_0	45.72136	101.88692
10	ap_1	45.82925	118.166695
10	ap_0	49.930725	142.40071
11	ap_1	44.744465	111.361755
11	ap_0	46.313942	117.64252
12	ap_1	46.4037	117.084656
12	ap_0	50.525066	127.543396
13	ap_1	48.02882	118.11169
13	ap_0	47.945194	133.16702
14	ap_1	49.610508	134.04526
14	ap_0	51.859844	158.96777
15	ap_1	47.135876	126.56822
15	ap_0	47.032578	139.05037
16	ap_1	48.63769	135.44464
16	ap_0	50.51939	161.66063
17	ap_1	47.92652	126.56709
17	ap_0	47.667984	139.51074
18	ap_1	49.430164	136.23734
18	ap_0	51.05303	158.23352
19	ap_1	45.90399	124.68248

19	ap_0	44.983875	141.18713
20	ap_1	47.44453	135.43338
20	ap_0	48.31151	164.65448
21	ap_1	40.02816	131.53696
21	ap_0	37.521584	112.47239
22	ap_1	41.651432	141.44342
22	ap_0	40.99506	140.71338
23	ap_1	43.386623	135.19809
23	ap_0	40.977234	118.68117
24	ap_1	45.09151	143.84808
24	ap_0	44.489674	130.48962
25	ap_1	37.38315	121.258286
25	ap_0	35.712116	84.04782
26	ap_1	39.209583	129.24141
26	ap_0	39.351852	116.748314
27	ap_1	34.47677	113.204765
27	ap_0	32.046177	69.34715
28	ap_1	36.418667	119.212425
28	ap_0	35.76194	100.99051
29	ap_1	31.269537	103.89276
29	ap_0	27.913979	72.25671
30	ap_1	33.290627	114.26634
30	ap_0	31.685982	87.378395
31	ap_1	31.372784	103.31033
31	ap_0	26.45952	73.2679
32	ap_1	33.37802	111.16432
32	ap_0	30.207794	92.95395
33	ap_1	30.221819	107.07186
33	ap_0	26.793446	70.59844
34	ap_1	30.115034	107.38652
34	ap_0	26.07968	70.69151
35	ap_1	31.959396	115.27312
35	ap_0	29.688211	92.21983
36	ap_1	29.786196	106.74832
36	ap_0	25.885147	75.69898
37	ap_1	31.449017	117.23895
37	ap_0	29.29302	96.1741
38	ap_1	28.601051	94.22362
38	ap_0	24.887062	70.6344
39	ap_1	26.896805	109.995224
39	ap_0	26.946367	82.29462
40	ap_1	28.252522	118.01107
40	ap_0	30.074127	102.0105

附录 2 test_set_1_3ap 结果表

3AP			
test_id	ap_id	predict seq_time	predict throughput
1	ap_2	28.09175	26.91417
1	ap_1	40.07205	46.027855
1	ap_0	35.99155	48.96458
2	ap_2	32.1087	28.271837
2	ap_1	38.5059	41.549652
2	ap_0	37.2843	40.284912
3	ap_2	29.6811	19.289963
3	ap_1	44.6339	44.18107
3	ap_0	37.77975	40.87954
4	ap_2	28.7342	38.93467
4	ap_1	41.64085	61.22185
4	ap_0	36.13745	34.69633
5	ap_2	32.0756	52.143578
5	ap_1	38.44295	59.32061
5	ap_0	36.75565	37.372253
6	ap_2	32.0715	10.760952
6	ap_1	40.5577	60.010014
6	ap_0	37.3809	48.453182
7	ap_2	31.44985	27.19534
7	ap_1	45.4957	58.353992
7	ap_0	28.9911	15.791686
8	ap_2	34.06825	44.761456
8	ap_1	44.1627	61.47015
8	ap_0	28.18455	10.841072
9	ap_2	25.56735	15.929193
9	ap_1	44.5996	62.941216
9	ap_0	26.30455	51.301323
10	ap_2	28.72185	23.800196
10	ap_1	41.7738	72.062126
10	ap_0	27.04595	46.848515
11	ap_2	25.8664	57.06632
11	ap_1	40.94505	60.126644
11	ap_0	17.3821	77.12653
12	ap_2	26.9028	73.32081
12	ap_1	39.01495	61.87312
12	ap_0	25.2884	11.099999
13	ap_2	26.39745	78.26382
13	ap_1	42.269	65.86893
13	ap_0	16.7864	90.36175
14	ap_2	27.7547	67.02359
14	ap_1	40.6628	61.687145

14	ap_0	24.458	114.621086
15	ap_2	26.0466	65.136055
15	ap_1	38.09735	61.492233
15	ap_0	17.1496	69.576515
16	ap_2	26.339	65.109505
16	ap_1	38.27935	61.13779
16	ap_0	17.0149	85.521416
17	ap_2	25.9039	55.8699
17	ap_1	35.9311	62.507687
17	ap_0	22.69435	55.094772
18	ap_2	33.23515	66.89184
18	ap_1	34.1224	45.778835
18	ap_0	19.16175	71.09455
19	ap_2	33.80785	61.49497
19	ap_1	31.12765	59.86477
19	ap_0	19.01995	74.67857
20	ap_2	23.44585	27.900341
20	ap_1	27.23085	44.67737
20	ap_0	18.1685	55.787518
21	ap_2	16.234	26.201258
21	ap_1	22.587	38.35563
21	ap_0	16.8691	49.98945
22	ap_2	17.60565	43.403534
22	ap_1	21.2756	45.993866
22	ap_0	15.414	92.567955
23	ap_2	17.1971	67.83358
23	ap_1	19.8899	46.957764
23	ap_0	17.68565	68.0371
24	ap_2	20.22525	60.02455
24	ap_1	24.69965	53.91091
24	ap_0	23.31885	41.141113
25	ap_2	15.85085	60.278362
25	ap_1	21.74255	42.0403
25	ap_0	16.34655	47.76435
26	ap_2	18.09545	45.92131
26	ap_1	22.13425	88.755936
26	ap_0	16.14315	65.61114
27	ap_2	20.3976	26.264856
27	ap_1	24.8749	101.75961
27	ap_0	24.33665	28.148623
28	ap_2	14.7104	56.965176
28	ap_1	19.95255	64.79973
28	ap_0	17.7349	36.29148
29	ap_2	20.0385	32.12172

29	ap_1	25.1985	38.47902
29	ap_0	23.59855	41.671345
30	ap_2	16.49425	46.341286
30	ap_1	19.85075	63.441734
30	ap_0	16.6102	75.21061
31	ap_2	20.29215	24.96141
31	ap_1	24.1839	69.73682
31	ap_0	23.7376	48.50093
32	ap_2	16.13665	58.313095
32	ap_1	19.11635	66.417534
32	ap_0	16.77935	53.467323
33	ap_2	21.4915	43.662624
33	ap_1	23.90595	81.656654
33	ap_0	24.0709	36.53366
34	ap_2	14.9421	44.31889
34	ap_1	17.55015	61.60092
34	ap_0	16.4376	41.300594
35	ap_2	20.0487	29.865652
35	ap_1	22.22615	43.76348
35	ap_0	23.9595	43.644783

附录3 test_set_2_2ap 结果表

2AP			
test_id	ap_id	predict nss	predict mcs
1	ap_1	2	8
1	ap_0	2	8
2	ap_1	2	8
2	ap_0	2	8
3	ap_1	2	8
3	ap_0	2	7
4	ap_1	2	8
4	ap_0	2	7
5	ap_1	2	7
5	ap_0	2	7
6	ap_1	2	7
6	ap_0	2	7
7	ap_1	2	8
7	ap_0	2	8
8	ap_1	2	8
8	ap_0	2	8
9	ap_1	2	9
9	ap_0	2	9

10	ap_1	2	7
10	ap_0	2	7
11	ap_1	2	7
11	ap_0	2	7
12	ap_1	2	7
12	ap_0	2	7
13	ap_1	2	8
13	ap_0	2	8
14	ap_1	2	8
14	ap_0	2	8
15	ap_1	2	8
15	ap_0	2	8
16	ap_1	2	8
16	ap_0	2	8
17	ap_1	2	8
17	ap_0	2	8
18	ap_1	2	7
18	ap_0	2	7
19	ap_1	2	7
19	ap_0	2	7
20	ap_1	2	8
20	ap_0	2	8
21	ap_1	2	8
21	ap_0	2	8
22	ap_1	2	8
22	ap_0	2	8
23	ap_1	2	7
23	ap_0	2	8
24	ap_1	2	7
24	ap_0	2	8
25	ap_1	2	9
25	ap_0	2	10
26	ap_1	2	9
26	ap_0	2	10
27	ap_1	2	8
27	ap_0	2	8
28	ap_1	2	8
28	ap_0	2	8
29	ap_1	2	7
29	ap_0	2	7
30	ap_1	2	7
30	ap_0	2	7
31	ap_1	2	7
31	ap_0	2	7

32	ap_1	2	7
32	ap_0	2	7

附录 4 test_set_2_3ap 结果表

3AP			
test_id	ap_id	predict nss	predict mcs
1	ap_2	2	5
1	ap_0	2	11
1	ap_1	2	10
2	ap_2	2	5
2	ap_0	2	11
2	ap_1	2	10
3	ap_2	2	5
3	ap_0	2	11
3	ap_1	2	9
4	ap_2	2	5
4	ap_0	2	11
4	ap_1	2	9
5	ap_2	2	6
5	ap_0	2	11
5	ap_1	2	11
6	ap_2	2	6
6	ap_0	2	11
6	ap_1	2	11
7	ap_2	2	5
7	ap_0	2	11
7	ap_1	2	9
8	ap_2	2	5
8	ap_0	2	11
8	ap_1	2	9
9	ap_2	2	6
9	ap_0	2	11
9	ap_1	2	9
10	ap_2	2	6
10	ap_0	2	11
10	ap_1	2	9
11	ap_2	2	5
11	ap_0	2	11
11	ap_1	2	11
12	ap_2	2	5
12	ap_0	2	11
12	ap_1	2	11

13	ap_2	2	5
13	ap_0	2	11
13	ap_1	2	11
14	ap_2	2	5
14	ap_0	2	11
14	ap_1	2	11
15	ap_2	2	6
15	ap_0	2	11
15	ap_1	2	11
16	ap_2	2	6
16	ap_0	2	11
16	ap_1	2	11
17	ap_2	2	6
17	ap_0	2	11
17	ap_1	2	11
18	ap_2	2	6
18	ap_0	2	11
18	ap_1	2	11
19	ap_2	2	6
19	ap_0	2	11
19	ap_1	2	11
20	ap_2	2	5
20	ap_0	2	11
20	ap_1	2	11
21	ap_2	2	6
21	ap_0	2	11
21	ap_1	2	11
22	ap_2	2	7
22	ap_0	2	11
22	ap_1	2	11
23	ap_2	2	7
23	ap_0	2	11
23	ap_1	2	9
24	ap_2	2	7
24	ap_0	2	11
24	ap_1	2	9
25	ap_2	2	8
25	ap_0	2	11
25	ap_1	2	11
26	ap_2	2	7
26	ap_0	2	11
26	ap_1	2	11
27	ap_2	2	7
27	ap_0	2	11

27	ap_1	2	11
28	ap_2	2	9
28	ap_0	2	10
28	ap_1	2	11
29	ap_2	2	10
29	ap_0	2	11
29	ap_1	2	11

附录 5: Q1_data_processing_3AP.py

```

import os
import pandas as pd
from sklearn.preprocessing import StandardScaler

directory_path = './'

files = [
    f for f in os.listdir(directory_path)
    if f.startswith('training_set_3ap') and f.endswith('.csv')
]

dataframes = []
for file in files:
    file_path = os.path.join(directory_path, file)
    df = pd.read_csv(file_path)
    if 'training_set_3ap_loc30' in file:
        df = df.drop(columns=['predict throughput', 'error%'], errors='ignore')
    dataframes.append(df)

combined_df = pd.concat(dataframes, ignore_index=True)

combined_df.to_excel('./output/01.combined_3ap.xlsx', index=False)
columns_to_remove = [
    'test_id', 'test_dur', 'pkt_len', 'ap_name', 'ap_mac', 'ap_id', 'sta_mac',
    'sta_id', 'other_air_time', 'ppdu_dur', 'num_ampdu'
]
combined_df = combined_df.drop(columns=columns_to_remove, errors='ignore')

processed_output_path = './output/02.processed_combined_3ap.xlsx'
combined_df.to_excel(processed_output_path, index=False)

print(f"Processed data saved to {processed_output_path}")

```

```

test_set_file = 'test_set_1_3ap.csv'
test_set_path = os.path.join(directory_path, test_set_file)
test_df = pd.read_csv(test_set_path)

print(f"Processing {test_set_file}")

columns_to_remove = [
    'test_id', 'test_dur', 'pkt_len', 'ap_name', 'ap_mac', 'ap_id', 'sta_mac',
    'sta_id', 'other_air_time', 'ppdu_dur', 'num_ampdu', 'predict seq_time',
    'error%', 'error%.1', 'predict throughput'
]
test_df = test_df.drop(columns=columns_to_remove, errors='ignore')

processed_test_output_path = './output/02.processed_test_set_3ap.xlsx'
test_df.to_excel(processed_test_output_path, index=False)

print(f"Processed test data saved to {processed_test_output_path}")

num_test_samples = len(test_df)

combined_df = pd.concat([combined_df, test_df], ignore_index=True)

print(f"Number of samples in test_df: {num_test_samples}")

encoded_df = pd.get_dummies(combined_df, columns=['loc_id', 'protocol', 'bss_id'])

one_hot_columns = [
    col for col in encoded_df.columns
    if col.startswith('loc_id_') or col.startswith('protocol_') or col.startswith('bss_id_')
]
encoded_df[one_hot_columns] = encoded_df[one_hot_columns].astype(int)

encoded_output_path = './output/03.one-hot_encoded_combined_3ap.xlsx'
encoded_df.to_excel(encoded_output_path, index=False)

print(f"Encoded data saved to {encoded_output_path}")

def dbm_to_mw(dbm):
    return 10 ** (dbm / 10)

columns_to_convert = ['pd', 'ed', 'nav', 'eirp']

```

```

for column in columns_to_convert:
    if column in encoded_df.columns:
        encoded_df[column] = encoded_df[column].apply(dbm_to_mw)

converted_output_path = './output/04.converted_mW_combined_3ap.xlsx'
encoded_df.to_excel(converted_output_path, index=False)

print(f"Converted data saved to {converted_output_path}")

sta_columns = [col for col in encoded_df.columns if col.startswith('sta_from_sta')]

for column in sta_columns:
    encoded_df[column] = encoded_df[column].apply(dbm_to_mw)

sta_columns = [col for col in encoded_df.columns if col.startswith('sta_from_sta')]

for column in sta_columns:
    for i in range(0, len(encoded_df), 3):
        group = encoded_df.iloc[i:i + 3]
        if len(group) < 3:
            continue
        empty_row_index = group[column].isna().idxmax()
        other_values = group[column].dropna()
        if len(other_values) == 2:
            mean_value = other_values.sum() / 2
            encoded_df.at[empty_row_index, column] = mean_value

sta_to_sta_output_path = './output/05.sta_to_sta_processed_combined_3ap.xlsx'
encoded_df.to_excel(sta_to_sta_output_path, index=False)

print(f"STA-to-STA processed data saved to {sta_to_sta_output_path}")

rssi_columns = [
    col for col in encoded_df.columns
    if col.endswith('rssi') and 'sta_from_sta' not in col
]

for column in rssi_columns:
    encoded_df[column] = encoded_df[column].apply(
        lambda x: [dbm_to_mw(float(val)) for val in x.strip('[]').split(',')]
        if isinstance(x, str) else x
    )

mean_columns = [col for col in rssi_columns if 'mean' in col]

```

```
max_columns = [col for col in rssi_columns if 'max' in col]
sum_columns = [col for col in rssi_columns if 'sum' in col]
```

```
for column in mean_columns:
```

```
    new_column = column + '_abv_nav'
    encoded_df[new_column] = encoded_df.apply(
        lambda row: sum(float(val) >= row['nav'] for val in row[column]) / len(row[column])
        if isinstance(row[column], list) else None,
        axis=1
    )
```

```
for column in max_columns:
```

```
    abv_pd_column = column + '_abv_pd'
    abv_ed_column = column + '_abv_ed'
    encoded_df[abv_pd_column] = encoded_df.apply(
        lambda row: sum(float(val) >= row['pd'] for val in row[column]) / len(row[column])
        if isinstance(row[column], list) else None,
        axis=1
    )
    encoded_df[abv_ed_column] = encoded_df.apply(
        lambda row: sum(float(val) >= row['ed'] for val in row[column]) / len(row[column])
        if isinstance(row[column], list) else None,
        axis=1
    )
```

```
for column in sum_columns:
```

```
    noise_rssi_column = column + '_noise_rssi'
```

```
def calculate_noise_rssi(row):
```

```
    if isinstance(row[column], list):
        valid_values = [float(val) for val in row[column] if float(val) < row['pd']]
        if valid_values:
            return sum(valid_values) / len(valid_values)
        return 0
    return None
```

```
encoded_df[noise_rssi_column] = encoded_df.apply(calculate_noise_rssi, axis=1)
```

```
encoded_df.drop(columns=rssi_columns, inplace=True)
```

```
newly_added_columns = [
```

```
    col for col in encoded_df.columns
    if col.endswith('_abv_nav') or col.endswith('_abv_pd') or col.endswith('_abv_ed') or
    col.endswith('_noise_rssi')
```

```
]
```

```
for column in newly_added_columns:
    for i in range(0, len(encoded_df), 3):
        group = encoded_df.iloc[i:i + 3]
        if len(group) < 3:
            continue
        empty_row_index = group[column].isna().idxmax()
        other_values = group[column].dropna()
        if len(other_values) == 2:
            mean_value = other_values.sum() / 2
            encoded_df.at[empty_row_index, column] = mean_value
```

```
final_excel_output_path = './output/06.final_processed_combined_3ap.xlsx'
encoded_df.to_excel(final_excel_output_path, index=False)
```

```
print(f"Final processed data saved to {final_excel_output_path}")
```

```
columns_to_standardize = [
    col for col in encoded_df.columns if col not in ['throughput', 'seq_time']
]
```

```
scaler = StandardScaler()
```

```
encoded_df[columns_to_standardize] = scaler.fit_transform(encoded_df[columns_to_standardize])
```

```
test_df = encoded_df.iloc[-num_test_samples:].copy()
encoded_df = encoded_df.iloc[:-num_test_samples]
```

```
separated_test_output_path = './output/08.separated_test_set_3ap.xlsx'
test_df.to_excel(separated_test_output_path, index=False)
separated_test_csv_output_path = './output/08.separated_test_set_3ap.csv'
test_df.to_csv(separated_test_csv_output_path, index=False)
```

```
print(f"Separated test data saved to {separated_test_output_path}")
print(f"Separated test data saved to {separated_test_csv_output_path}")
```

```
standardized_output_path = './output/07.standardized_combined_3ap.xlsx'
encoded_df.to_excel(standardized_output_path, index=False)
standardized_csv_output_path = './output/07.standardized_combined_3ap.csv'
encoded_df.to_csv(standardized_csv_output_path, index=False)
```

```
print(f"Standardized data saved to {standardized_output_path}")
print(f"Standardized data saved to {standardized_csv_output_path}")
```

附录 6: Q1_randomforest_analysis_3AP.py

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.read_csv('./output/07.standardized_combined_3ap.csv')
data = data.drop(columns=['throughput'])
columns_to_drop = [col for col in data.columns if 'sta_from_sta' in col or col in ['per', 'nss', 'mcs']]
data = data.drop(columns=columns_to_drop)
X = data.drop(columns=['seq_time'])
y = data['seq_time']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.05, random_state=42)

rf = RandomForestRegressor(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')

feature_importances = rf.feature_importances_
features = X.columns

importance_df = pd.DataFrame({'Feature': features, 'Importance': feature_importances})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
importance_df.to_csv('./output/feature_importances.csv', index=False)

top_20_features = importance_df.head(20)

plt.figure(figsize=(10, 8))
sns.barplot(x='Importance', y='Feature', data=top_20_features)
plt.title('Random Forest Top 20 3AP Feature Importances')
plt.tight_layout()
plt.show()

new_test_data = pd.read_csv('./output/08.separated_test_set_3ap.csv')
new_test_data = new_test_data.drop(columns=['throughput'])
columns_to_drop = [col for col in new_test_data.columns if 'sta_from_sta' in col or col in ['per', 'nss', 'mcs']]
```



```

new_test_data = new_test_data.drop(columns=columns_to_drop)
X_new_test = new_test_data.drop(columns=['seq_time'])

new_predictions = rf.predict(X_new_test)
new_test_data['seq_time'] = new_predictions
new_test_data['seq_time'].to_csv('./output/Q1_result.csv', index=False)

```

附录 7: Q2_cnn_3AP.py

```

import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset, random_split
import sys

```

```

predict_flag = 0

```

```

class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=(3, 3))
        self.pool = nn.MaxPool2d(kernel_size=(2, 2))
        self.conv2 = nn.Conv2d(32, 64, kernel_size=(3, 3))
        self.dropout = nn.Dropout(p=0.5)
        self.fc1_input_size = 64 * 133 * 2
        self.fc1 = nn.Linear(self.fc1_input_size, 64)
        self.fc2 = nn.Linear(64, 2)

    def forward(self, x):
        x = self.pool(nn.functional.relu(self.conv1(x)))
        x = self.pool(nn.functional.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.dropout(x)
        x = nn.functional.relu(self.fc1(x))
        x = self.fc2(x)
        return x

```

```

def process_string(s):
    s = s.strip('[]')

```

```

return [float(x) for x in s.split(',')]

if predict_flag == 1:
    model = CNNModel()
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model.load_state_dict(torch.load('best_cnn_model_nss_mcs.pth'))
    model.to(device)
    model.eval()

    test_df = pd.read_csv('./output/Q2_3AP/07.test_set.csv')
    test_df = test_df.drop(columns=['nss', 'mcs', 'phy_rate'])
    for col in test_df.columns:
        test_df[col] = test_df[col].apply(process_string)

    X_test_list = []
    for index, row in test_df.iterrows():
        feature_arrays = [np.array(row[col]) for col in test_df.columns]
        feature_stack = np.stack(feature_arrays, axis=0)
        X_test_list.append(feature_stack)

    X_test = np.array(X_test_list)
    X_test = X_test.transpose(0, 2, 1)
    X_test = X_test[..., np.newaxis]
    X_test = X_test.transpose(0, 3, 1, 2)
    X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)

    with torch.no_grad():
        predictions = model(X_test_tensor)

    predictions = predictions.cpu().numpy()
    predictions = np.round(predictions)
    result_df = pd.DataFrame(predictions, columns=['nss', 'mcs'])
    result_df.loc[result_df['nss'] > 2, 'nss'] = 2
    result_df.loc[result_df['nss'] > 2, 'mcs'] = 11
    result_df.loc[result_df['mcs'] > 11, 'mcs'] = 11

    for i in range(0, len(result_df), 3):
        if i + 2 < len(result_df):
            result_df.iloc[i + 1], result_df.iloc[i + 2] = result_df.iloc[i + 2].copy(),
result_df.iloc[i + 1].copy()

result_df = result_df.astype(int)
result_df.to_csv('./output/Q2_3AP/Q2_result_AP3.csv', index=False)

```

```

sys.exit()

df = pd.read_csv('./output/Q2_3AP/06.transformed_data.csv')
df = df.drop(columns=['phy_rate'])
feature_columns = df.columns.drop(['nss', 'mcs'])
target_columns = ['nss', 'mcs']

for col in feature_columns:
    df[col] = df[col].apply(process_string)

df = df[df['nss'] != 0]

X_list = []
for index, row in df.iterrows():
    feature_arrays = [np.array(row[col]) for col in feature_columns]
    feature_stack = np.stack(feature_arrays, axis=0)
    X_list.append(feature_stack)

X = np.array(X_list)
X = X.transpose(0, 2, 1)
X = X[..., np.newaxis]
X = X.transpose(0, 3, 1, 2)
y = df[target_columns].values

X_tensor = torch.tensor(X, dtype=torch.float32)
y_tensor = torch.tensor(y, dtype=torch.float32)
dataset = TensorDataset(X_tensor, y_tensor)

train_size = int(0.95 * len(dataset))
test_size = len(dataset) - train_size
train_dataset, test_dataset = random_split(dataset, [train_size, test_size])

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

model = CNNModel()
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.0001)
num_epochs = 500

for epoch in range(num_epochs):

```

```

model.train()
for inputs, targets in train_loader:
    inputs, targets = inputs.to(device), targets.to(device)
    optimizer.zero_grad()
    outputs = model(inputs)
    if torch.isnan(outputs).any() or torch.isinf(outputs).any():
        print("Output contains NaN or Inf")
        break
    loss = criterion(outputs, targets)
    loss.backward()
    optimizer.step()

print(f'Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}')
with open('./output/Q2_3AP/training_loss_log.txt', 'a') as f:
    f.write(f'Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}\n')

if (epoch + 1) % 20 == 0:
    model.eval()
    test_loss = 0
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, targets in test_loader:
            inputs, targets = inputs.to(device), targets.to(device)
            outputs = model(inputs)
            test_loss += criterion(outputs, targets).item()
            predicted = torch.round(outputs)
            correct += (predicted == targets).sum().item()
            total += targets.numel()

    test_loss /= len(test_loader)
    accuracy = correct / total
    print(f'Test Loss: {test_loss:.4f}, Accuracy: {accuracy:.4f}')

    if 'best_accuracy' not in locals():
        best_accuracy = 0
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        torch.save(model.state_dict(), 'best_cnn_model_nss_mcs.pth')
        print(f'New best accuracy: {best_accuracy:.4f}. Model saved.')

```

附录 8: Q2_data_processing_3AP.py

```
import os
import numpy as np
import pandas as pd
from scipy.interpolate import Akima1DInterpolator

output_path = './output/Q2_3AP'
directory_path = './'

files = [f for f in os.listdir(directory_path) if f.startswith('training_set_3ap') and f.endswith('.csv')]

dataframes = []
for file in files:
    file_path = os.path.join(directory_path, file)
    df = pd.read_csv(file_path)
    if 'training_set_3ap_loc30' in file:
        df = df.drop(columns=['predict throughput', 'error%'], errors='ignore')
    dataframes.append(df)

combined_df = pd.concat(dataframes, ignore_index=True)
error_processed_df = pd.read_csv('./error_processed_test_set_2_3ap.csv')
num_rows_error_processed = len(error_processed_df)
print(f"Number of rows in error processed test set: {num_rows_error_processed}")

combined_df = pd.concat([combined_df, error_processed_df], ignore_index=True)
columns_to_drop = [col for col in combined_df.columns if col.startswith('sta_from_sta')]
combined_df = combined_df.drop(columns=columns_to_drop)

output_file_path = os.path.join(output_path, '01.remove_sta_from_sta_data.xlsx')
combined_df.to_excel(output_file_path, index=False)

def akima_interpolate(arr, target_length=540):
    x = np.arange(len(arr))
    interpolator = Akima1DInterpolator(x, arr)
    x_new = np.linspace(0, len(arr) - 1, target_length)
    return interpolator(x_new)

for col in combined_df.columns:
    if col.endswith('rssi'):
        combined_df[col] = combined_df[col].apply(lambda x: [-100] if pd.isna(x) else list(map(int, x.strip('[]').split(','))))
```

```

combined_df[col] = combined_df[col].apply(lambda x: x * 2 if len(x) == 1 else x)
combined_df[col] = combined_df[col].apply(lambda x: akima_interpolate(x, 540))

output_file_path_interpolated = os.path.join(output_path, '02.interpolated_rssi_data.xlsx')
print(f"Processed data saved to {output_file_path_interpolated}")

for agg_func in ['sum', 'mean', 'max']:
    for i in range(2):
        combined_df[f'sta_from_ap_{agg_func}_sinr_{i}'] = np.nan
for agg_func in ['sum', 'mean', 'max']:
    for i in range(2):
        combined_df[f'sta_to_ap_{agg_func}_sinr_{i}'] = np.nan

output_file_path_features = os.path.join(output_path, '03.added_features_data.xlsx')
print(f"Processed data with new features saved to {output_file_path_features}")

for index, row in combined_df.iterrows():
    sta_id = row['bss_id']
    other_sta_ids = [i for i in range(3) if i != sta_id]
    for agg_func in ['sum', 'mean', 'max']:
        sta_from_ap_current = np.array(row[f'sta_from_ap_{sta_id}_{agg_func}_ant_rssi'])
        sta_from_ap_other_0 = np.array(row[f'sta_from_ap_{other_sta_ids[0]}_{agg_func}_ant_rssi'])
        sta_from_ap_other_1 = np.array(row[f'sta_from_ap_{other_sta_ids[1]}_{agg_func}_ant_rssi'])
        sinr_0 = (sta_from_ap_current - sta_from_ap_other_0).astype(int)
        sinr_1 = (sta_from_ap_current - sta_from_ap_other_1).astype(int)
        combined_df.at[index, f'sta_from_ap_{agg_func}_sinr_0'] = str(sinr_0.tolist())
        combined_df.at[index, f'sta_from_ap_{agg_func}_sinr_1'] = str(sinr_1.tolist())

for agg_func in ['sum', 'mean', 'max']:
    for i in range(2):
        col_name = f'sta_from_ap_{agg_func}_sinr_{i}'
        combined_df[col_name] = combined_df[col_name].apply(lambda x: list(map(int,
x.strip('[]').split(','))) if pd.notna(x) else x)

for index, row in combined_df.iterrows():
    ap_id = row['bss_id']
    other_ap_ids = [i for i in range(3) if i != ap_id]
    for agg_func in ['sum', 'mean', 'max']:
        sta_to_ap_current = np.array(row[f'sta_to_ap_{ap_id}_{agg_func}_ant_rssi'])
        if ap_id == 2:
            sta_to_ap_other_0 = np.array(combined_df.iloc[index
1][f'sta_to_ap_{ap_id}_{agg_func}_ant_rssi'])
            sta_to_ap_other_1 = np.array(combined_df.iloc[index
2][f'sta_to_ap_{ap_id}_{agg_func}_ant_rssi'])

```

```

        elif ap_id == 1:
            sta_to_ap_other_0 = np.array(combined_df.iloc[index
1][f'sta_to_ap_{ap_id}_{agg_func}_ant_rssi'])
            sta_to_ap_other_1 = np.array(combined_df.iloc[index
1][f'sta_to_ap_{ap_id}_{agg_func}_ant_rssi'])
        elif ap_id == 0:
            sta_to_ap_other_0 = np.array(combined_df.iloc[index
2][f'sta_to_ap_{ap_id}_{agg_func}_ant_rssi'])
            sta_to_ap_other_1 = np.array(combined_df.iloc[index
1][f'sta_to_ap_{ap_id}_{agg_func}_ant_rssi'])
            sinr_0 = (sta_to_ap_current - sta_to_ap_other_0).astype(int)
            sinr_1 = (sta_to_ap_current - sta_to_ap_other_1).astype(int)
            combined_df.at[index, f'sta_to_ap_{agg_func}_sinr_0'] = str(sinr_0.tolist())
            combined_df.at[index, f'sta_to_ap_{agg_func}_sinr_1'] = str(sinr_1.tolist())

for agg_func in ['sum', 'mean', 'max']:
    for i in range(2):
        col_name = f'sta_to_ap_{agg_func}_sinr_{i}'
        combined_df[col_name] = combined_df[col_name].apply(lambda x: list(map(int,
x.strip('[]').split(','))) if pd.notna(x) else x)

columns_to_drop_rssi = [col for col in combined_df.columns if 'rssi' in col]
combined_df = combined_df.drop(columns=columns_to_drop_rssi)
columns_to_drop_additional = [
    'seq_time', 'throughput', 'other_air_time', 'sta_id', 'ppdu_dur', 'num_ampdu', 'per', 'test_dur', 'pkt_len',
    'pd',
    'ed', 'nav', 'eirp', 'protocol', 'test_id', 'ap_name', 'ap_mac', 'sta_mac', 'bss_id', 'loc_id'
]
combined_df = combined_df.drop(columns=columns_to_drop_additional)

mcs_to_phy_rate = {
    0: [8.6, 17.2],
    1: [17.2, 34.4],
    2: [25.8, 51.6],
    3: [34.4, 68.8],
    4: [51.6, 103.2],
    5: [68.8, 137.6],
    6: [77.4, 154.9],
    7: [86.0, 172.1],
    8: [103.2, 206.5],
    9: [114.7, 229.4],
    10: [129.0, 258.1],
    11: [143.4, 286.8]
}

```

```

def convert_to_phy_rate(mcs, nss):
    if pd.isna(mcs) or pd.isna(nss):
        return np.nan
    mcs = int(mcs)
    nss = int(nss)
    if mcs == 0 and nss == 0:
        return 0
    if mcs in mcs_to_phy_rate and nss in [1, 2]:
        return mcs_to_phy_rate[mcs][nss - 1]
    return np.nan

combined_df['phy_rate'] = combined_df.apply(lambda row: convert_to_phy_rate(row['mcs'], row['nss']),
axis=1)

output_file_path_filled = os.path.join(output_path, '04.filled_sta_from_ap_data.xlsx')
combined_df.to_excel(output_file_path_filled, index=False)

print(f"Processed data with filled sta_from_ap saved to {output_file_path_filled}")

one_hot_encoded = pd.get_dummies(combined_df['ap_id'], prefix='ap_id').astype(int)
combined_df = pd.concat([combined_df, one_hot_encoded], axis=1)
combined_df = combined_df.drop(columns=['ap_id'])
phy_rate_col = combined_df.pop('phy_rate')
combined_df['phy_rate'] = phy_rate_col

output_file_path_one_hot = os.path.join(output_path, '05.one_hot_encoded_data.xlsx')
combined_df.to_excel(output_file_path_one_hot, index=False)

print(f"Processed data with one hot encoding saved to {output_file_path_one_hot}")

columns_to_transform = ['ap_id_ap_0', 'ap_id_ap_1', 'ap_id_ap_2']
for column in columns_to_transform:
    combined_df[column] = combined_df[column].apply(lambda x: [x] * 540)

output_file_path_transformed = os.path.join(output_path, '06.transformed_data.xlsx')
combined_df.to_excel(output_file_path_transformed, index=False)

print(f"Transformed data saved to {output_file_path_transformed}")

test_set = combined_df.iloc[-num_rows_error_processed:]
training_set = combined_df.iloc[:-num_rows_error_processed]

```



```

output_file_path_training_set = os.path.join(output_path, '07.training_set.xlsx')
output_file_path_test_set = os.path.join(output_path, '07.test_set.xlsx')
training_set.to_excel(output_file_path_training_set, index=False)
test_set.to_excel(output_file_path_test_set, index=False)

print(f"Training set saved to {output_file_path_training_set}")
print(f"Test set saved to {output_file_path_test_set}")

output_file_path_training_set_csv = os.path.join(output_path, '07.training_set.csv')
output_file_path_test_set_csv = os.path.join(output_path, '07.test_set.csv')
training_set.to_csv(output_file_path_training_set_csv, index=False)
test_set.to_csv(output_file_path_test_set_csv, index=False)

print(f"Training set saved to {output_file_path_training_set_csv}")
print(f"Test set saved to {output_file_path_test_set_csv}")

```

附录 9: Q2_error_correcting.py

```

import pandas as pd

df = pd.read_csv('./test_set_2_3ap.csv')

columns_to_check = [
    'sta_from_ap_1_max_ant_rssi',
    'sta_from_ap_1_mean_ant_rssi',
    'sta_from_ap_2_max_ant_rssi',
    'sta_from_ap_2_mean_ant_rssi'
]

df.loc[df['sta_id'] == 'sta_1', columns_to_check] = "[-100]"
df = df.sort_values(by=['test_id', 'bss_id'], ascending=[True, False])
df = df.iloc[:, :-3]
df.to_csv('error_processed_test_set_2_3ap.csv', index=False)

```

附录 10: readData2.py

```

import pandas as pd
import numpy as np
from scipy.interpolate import Akima1DInterpolator
import ast

```

```

def pad_lists_and_min(series1, series2, pad_value=-100):
    # 定义目标长度
    target_length = 540

    def pad_and_interpolate(series):
        # 对每个列表进行插值
        interpolated_series = []
        for lst in series:
            if len(lst) == 1:
                # 如果列表只有一个元素，直接复制
                interpolated_lst = np.array([lst[0]] * target_length)
            elif len(lst) < target_length:
                x = np.arange(len(lst))
                interpolator = Akima1DInterpolator(x, lst)
                x_new = np.linspace(0, len(lst) - 1, target_length)
                interpolated_lst = interpolator(x_new)
            else:
                interpolated_lst = np.array(lst[:target_length])
            interpolated_series.append(interpolated_lst)

        return interpolated_series

    # 对两个系列进行插值
    interpolated_series1 = pad_and_interpolate(series1)
    interpolated_series2 = pad_and_interpolate(series2)

    # 计算两个列表中每个元素的差值
    result_series = [list(np.array(lst1) - np.array(lst2)) for lst1, lst2 in zip(interpolated_series1,
interpolated_series2)]

    # 将每一行复制并新增为下一行
    doubled_result_series = []
    for row in result_series:
        doubled_result_series.append(row)
        doubled_result_series.append(row.copy())
    # 将列表输出为 csv 文件
    #pd.DataFrame(result_series).to_csv('result_series.csv',index=False)
    return pd.Series(doubled_result_series)

# 读取 CSV 文件
def read_2ap_for_question2(file_path):
    data = pd.read_csv(file_path)
    data = data.drop(columns=['seq_time', 'throughput', 'other_air_time', 'ppdu_dur', 'num_ampdu', 'per',

```

```

'mcs', 'nss'])
    data = data.drop(columns=['test_dur','pkt_len','pd','ed','nav','eirp'])
    data = data.drop(columns=['protocol'])
    data = data.drop(columns=['test_id', 'ap_name', 'ap_mac', 'sta_mac', 'ap_id', 'bss_id','loc_id'])
    data = data.drop(columns=['ap_from_ap_0_sum_ant_rssi','ap_from_ap_0_max_ant_rssi','ap_from_ap_0_mean_ant_rssi',
    'ap_from_ap_1_sum_ant_rssi','ap_from_ap_1_max_ant_rssi','ap_from_ap_1_mean_ant_rssi',
    'sta_from_sta_0_rssi','sta_from_sta_1_rssi'])

# 将列表中的内容转换为数值 list 类型
columns_to_convert = [
    'sta_to_ap_0_sum_ant_rssi', 'sta_to_ap_0_max_ant_rssi', 'sta_to_ap_0_mean_ant_rssi',
    'sta_to_ap_1_sum_ant_rssi', 'sta_to_ap_1_max_ant_rssi', 'sta_to_ap_1_mean_ant_rssi',
    'sta_from_ap_0_sum_ant_rssi', 'sta_from_ap_0_max_ant_rssi', 'sta_from_ap_0_mean_ant_rssi',
    'sta_from_ap_1_sum_ant_rssi', 'sta_from_ap_1_max_ant_rssi', 'sta_from_ap_1_mean_ant_rssi'
]

for column in columns_to_convert:
    string = data[column].apply(lambda x:x[1:-1])
    vec = string.apply(lambda x:x.split(','))
    # 将字符串列表转换为浮点数列表
    vec = vec.apply(lambda x:[float(i) for i in x])
    data[column] = vec

# print(type(data['sta_to_ap_0_sum_ant_rssi']))

data0 = data[data['sta_id'] == 'sta_0']
data1 = data[data['sta_id'] == 'sta_1']

data0 = data0.drop(columns=['sta_id'])
data1 = data1.drop(columns=['sta_id'])

data0['id_0'] = 1
data1['id_0'] = 0
data0['id_1'] = 0
data1['id_1'] = 1

data0.index = data0.index-1

data0['sta0_sum_sinr'] = data1['sta0_sum_sinr']
pad_lists_and_min(data0['sta_from_ap_0_sum_ant_rssi'],data0['sta_from_ap_1_sum_ant_rssi'])
data0['sta1_sum_sinr'] = data1['sta1_sum_sinr']

```

```

pad_lists_and_min(data1['sta_from_ap_1_sum_ant_rssi'],data1['sta_from_ap_0_sum_ant_rssi'])
    data0['sta0_mean_sinr'] = data1['sta0_mean_sinr'] =
pad_lists_and_min(data0['sta_from_ap_0_mean_ant_rssi'],data0['sta_from_ap_1_mean_ant_rssi'])
    data0['sta1_mean_sinr'] = data1['sta1_mean_sinr'] =
pad_lists_and_min(data1['sta_from_ap_1_mean_ant_rssi'],data1['sta_from_ap_0_mean_ant_rssi'])
    data0['sta0_max_sinr'] = data1['sta0_max_sinr'] =
pad_lists_and_min(data0['sta_from_ap_0_max_ant_rssi'],data0['sta_from_ap_1_max_ant_rssi'])
    data0['sta1_max_sinr'] = data1['sta1_max_sinr'] =
pad_lists_and_min(data1['sta_from_ap_1_max_ant_rssi'],data1['sta_from_ap_0_max_ant_rssi'])
    data0['ap0_sum_sinr'] = data1['ap0_sum_sinr'] =
pad_lists_and_min(data0['sta_to_ap_0_sum_ant_rssi'],data1['sta_to_ap_0_sum_ant_rssi'])
    data0['ap1_sum_sinr'] = data1['ap1_sum_sinr'] =
pad_lists_and_min(data1['sta_to_ap_1_sum_ant_rssi'],data0['sta_to_ap_1_sum_ant_rssi'])
    data0['ap0_mean_sinr'] = data1['ap0_mean_sinr'] =
pad_lists_and_min(data0['sta_to_ap_0_mean_ant_rssi'],data1['sta_to_ap_0_mean_ant_rssi'])
    data0['ap1_mean_sinr'] = data1['ap1_mean_sinr'] =
pad_lists_and_min(data1['sta_to_ap_1_mean_ant_rssi'],data0['sta_to_ap_1_mean_ant_rssi'])
    data0['ap0_max_sinr'] = data1['ap0_max_sinr'] =
pad_lists_and_min(data0['sta_to_ap_0_max_ant_rssi'],data1['sta_to_ap_0_max_ant_rssi'])
    data0['ap1_max_sinr'] = data1['ap1_max_sinr'] =
pad_lists_and_min(data1['sta_to_ap_1_max_ant_rssi'],data0['sta_to_ap_1_max_ant_rssi'])

```

```

data0.index = data0.index+1

```

```

data = pd.concat([data0, data1]).sort_index()

```

```

columns_to_drop = [
    'sta_to_ap_0_sum_ant_rssi', 'sta_to_ap_0_max_ant_rssi', 'sta_to_ap_0_mean_ant_rssi',
    'sta_to_ap_1_sum_ant_rssi', 'sta_to_ap_1_max_ant_rssi', 'sta_to_ap_1_mean_ant_rssi',
    'sta_from_ap_0_sum_ant_rssi', 'sta_from_ap_0_max_ant_rssi', 'sta_from_ap_0_mean_ant_rssi',
    'sta_from_ap_1_sum_ant_rssi', 'sta_from_ap_1_max_ant_rssi', 'sta_from_ap_1_mean_ant_rssi'
]

```

```

data = data.drop(columns=columns_to_drop)

```

```

return data

```

```

if __name__ == '__main__':

```

```

    file_path = 'training_set_2ap_loc0_nav86.csv'

```

```

    data = read_2ap_for_question2(file_path)

```

```

    #输出为 csv

```

```

    data.to_csv('test.csv',index=False)

```

```

    print(data)

```

```

    print(data.head())

```

```

    print(data.shape)

```

附录 11: readData.py

```
import pandas as pd
```

```
# Function to read and display the csv file
```

```
def read_and_display_2ap_csv(file_path):
```

```
    data = pd.read_csv(file_path)
```

```
    # 将 loc3 和 loc4 转化为 unknown
```

```
    data['loc_id'] = data['loc_id'].replace({'loc3': 'unknown', 'loc4': 'unknown'})
```

```
    # 进行 One-Hot 编码
```

```
    data['loc_id'] = data['loc_id'].astype('category')
```

```
    data['loc_id'] = data['loc_id'].cat.set_categories(['loc0', 'loc1', 'loc2', 'unknown'])
```

```
    data = pd.get_dummies(data, columns=['loc_id'], prefix='loc')
```

```
    data = pd.get_dummies(data, columns=['protocol'], prefix='protocol')
```

```
    data = pd.get_dummies(data, columns=['bss_id'], prefix='bss_id')
```

```
    columns_to_convert = ['loc_loc0', 'loc_loc1', 'loc_loc2', 'loc_unknown', 'protocol_tcp',  
'protocol_udp', 'bss_id_0', 'bss_id_1']
```

```
    data[columns_to_convert] = data[columns_to_convert].astype(int)
```

```
    # 将信号强度转化为线性数据
```

```
    data['eirp'] = 10 ** (data['eirp'] / 10)
```

```
    # 去除非数值列和目标列
```

```
    features = data.drop(columns=['test_id', 'ap_name', 'ap_mac', 'sta_mac', 'sta_id'])
```

```
    # 输出为 csv 文件
```

```
    features.to_csv('features.csv', index=False)
```

```
    # 将缺省数据填充为 0
```

```
    features.fillna(0, inplace=True)
```

```
    # 根据 ap_id 列的值分割数据
```

```
    data_ap_1 = features[features['ap_id'] == 'ap_1']
```

```
    data_ap_0 = features[features['ap_id'] == 'ap_0']
```

```
    temp_data1 = data_ap_1.drop(columns=['ap_id', 'ap_from_ap_0_sum_ant_rssi',  
'ap_from_ap_0_max_ant_rssi', 'ap_from_ap_0_mean_ant_rssi', 'ap_from_ap_1_sum_ant_rssi',  
'ap_from_ap_1_max_ant_rssi', 'ap_from_ap_1_mean_ant_rssi', 'sta_to_ap_0_max_ant_rssi', 'sta_to_ap_1_max_a  
nt_rssi', 'sta_to_ap_0_mean_ant_rssi', 'sta_to_ap_1_mean_ant_rssi', 'sta_from_ap_0_max_ant_rssi', 'sta_from_a  
p_1_max_ant_rssi', 'sta_from_ap_0_mean_ant_rssi', 'sta_from_ap_1_mean_ant_rssi'])
```

```
    temp_data0 = data_ap_0.drop(columns=['ap_id', 'ap_from_ap_0_sum_ant_rssi',  
'ap_from_ap_0_max_ant_rssi', 'ap_from_ap_0_mean_ant_rssi', 'ap_from_ap_1_sum_ant_rssi',  
'ap_from_ap_1_max_ant_rssi', 'ap_from_ap_1_mean_ant_rssi', 'sta_to_ap_0_max_ant_rssi', 'sta_to_ap_1_max_a  
nt_rssi', 'sta_to_ap_0_mean_ant_rssi', 'sta_to_ap_1_mean_ant_rssi', 'sta_from_ap_0_max_ant_rssi', 'sta_from_a
```

```

p_1_max_ant_rssi','sta_from_ap_0_mean_ant_rssi','sta_from_ap_1_mean_ant_rssi'])
    data1
temp_data1.drop(columns=['sta_to_ap_0_sum_ant_rssi','sta_to_ap_1_sum_ant_rssi','sta_from_ap_0_sum_a
nt_rssi','sta_from_ap_1_sum_ant_rssi','sta_from_sta_0_rssi','sta_from_sta_1_rssi'])
    data0
temp_data0.drop(columns=['sta_to_ap_0_sum_ant_rssi','sta_to_ap_1_sum_ant_rssi','sta_from_ap_0_sum_a
nt_rssi','sta_from_ap_1_sum_ant_rssi','sta_from_sta_0_rssi','sta_from_sta_1_rssi'])

# 将 ap_from_ap_1_max_ant_rssi 的值转化为数组类型
string_0 = data_ap_0['ap_from_ap_1_max_ant_rssi'].apply(lambda x: x[1:-1])
vec0 = string_0.str.split(',')

# 将 ap_from_ap_0_max_ant_rssi 的值转化为数组类型
string_1 = data_ap_1['ap_from_ap_0_max_ant_rssi'].apply(lambda x: x[1:-1])
vec1 = string_1.str.split(',')

# 确保 vec1 和 vec0 的索引与 data1 和 data0 对齐
vec1.index = data1.index
vec0.index = data0.index

if 'ap_from_ap_0_max_ant_rssi' in data_ap_1.columns and 'pd' in data_ap_1.columns:
    # 将 vec1 中每一行里值大于 pd 的个数赋给 num
    num = vec1.apply(lambda x: sum(float(i) >= data_ap_1['pd']).iloc[min(idx, len(data_ap_1['pd'])-
1)] for idx, i in enumerate(x)))
    total1 = vec1.apply(len)
    data1['ap0_above_pd'] = num / total1
    num.index = num.index+1
    total1.index = total1.index+1
    data0['ap0_above_pd'] = num / total1

if 'ap_from_ap_1_max_ant_rssi' in data_ap_0.columns and 'pd' in data_ap_0.columns:
    # 将 vec0 中每一行里值大于 pd 的个数赋给 num
    num = vec0.apply(lambda x: sum(float(i) >= data_ap_0['pd']).iloc[min(idx, len(data_ap_0['pd'])-
1)] for idx, i in enumerate(x)))
    total0 = vec0.apply(len)
    data0['ap1_above_pd'] = num / total0
    num.index = num.index-1
    total0.index = total0.index-1
    data1['ap1_above_pd'] = num / total0

if 'ap_from_ap_0_max_ant_rssi' in data_ap_1.columns and 'ed' in data_ap_1.columns:
    # 将 vec1 中每一行里值大于 ed 的个数赋给 num

```

```

        num = vec1.apply(lambda x: sum(float(i) >= data_ap_1['ed'].iloc[min(idx, len(data_ap_1['ed']))-1]) for idx, i in enumerate(x)))
        total1 = vec1.apply(len)
        data1['ap0_above_ed'] = num / total1
        num.index = num.index+1
        total1.index = total1.index+1
        data0['ap0_above_ed'] = num / total1

if 'ap_from_ap_1_max_ant_rssi' in data_ap_0.columns and 'ed' in data_ap_0.columns:
    # 将 vec0 中每一行里值大于 ed 的个数赋给 num
    num = vec0.apply(lambda x: sum(float(i) >= data_ap_0['ed'].iloc[min(idx, len(data_ap_0['ed']))-1]) for idx, i in enumerate(x)))
    total0 = vec0.apply(len)
    data0['ap1_above_ed'] = num / total0
    num.index = num.index-1
    total0.index = total0.index-1
    data1['ap1_above_ed'] = num / total0

# 将 ap_from_ap_1_mean_ant_rssi 的值转化为数组类型
string_0 = data_ap_0['ap_from_ap_1_mean_ant_rssi'].apply(lambda x: x[1:-1])
vec0 = string_0.str.split(',')
total0 = vec0.apply(len)

# 将 ap_from_ap_0_mean_ant_rssi 的值转化为数组类型
string_1 = data_ap_1['ap_from_ap_0_mean_ant_rssi'].apply(lambda x: x[1:-1])
vec1 = string_1.str.split(',')
total1 = vec1.apply(len)

if 'ap_from_ap_0_mean_ant_rssi' in data_ap_1.columns and 'nav' in data_ap_1.columns:
    # 将 vec1 中每一行里值大于 nav 的个数赋给 num
    num = vec1.apply(lambda x: sum(float(i) >= data_ap_1['nav'].iloc[min(idx, len(data_ap_1['nav'])-1]) for idx, i in enumerate(x)))
    data1['ap0_above_nav'] = num / total1
    num.index = num.index+1
    total1.index = total1.index+1
    data0['ap0_above_nav'] = num / total1
if 'ap_from_ap_1_mean_ant_rssi' in data_ap_0.columns and 'nav' in data_ap_0.columns:
    # 将 vec0 中每一行里值大于 nav 的个数赋给 num
    num = vec0.apply(lambda x: sum(float(i) >= data_ap_0['nav'].iloc[min(idx, len(data_ap_0['nav'])-1]) for idx, i in enumerate(x)))
    data0['ap1_above_nav'] = num / total0
    num.index = num.index-1
    total0.index = total0.index-1
    data1['ap1_above_nav'] = num / total0

```

```

# 将 sta_to_ap_0_max_ant_rssi 的值转化为数组类型
string_0to0 = data_ap_0['sta_to_ap_0_max_ant_rssi'].apply(lambda x: x[1:-1])
vec0to0 = string_0to0.str.split(',')
total0to0 = vec0to0.apply(len)

# 将 sta_to_ap_0_max_ant_rssi 的值转化为数组类型
string_1to0 = data_ap_1['sta_to_ap_0_max_ant_rssi'].apply(lambda x: x[1:-1])
vec1to0 = string_1to0.str.split(',')
total1to0 = vec1to0.apply(len)

# 将 sta_to_ap_1_max_ant_rssi 的值转化为数组类型
string_0to1 = data_ap_0['sta_to_ap_1_max_ant_rssi'].apply(lambda x: x[1:-1])
vec0to1 = string_0to1.str.split(',')
total0to1 = vec0to1.apply(len)

# 将 sta_to_ap_1_max_ant_rssi 的值转化为数组类型
string_1to1 = data_ap_1['sta_to_ap_1_max_ant_rssi'].apply(lambda x: x[1:-1])
vec1to1 = string_1to1.str.split(',')
total1to1 = vec1to1.apply(len)

if 'sta_to_ap_0_max_ant_rssi' in data_ap_1.columns and 'pd' in data_ap_1.columns:
    # 将 vec1to0 中每一行里值大于 pd 的个数赋给 num
    num = vec1to0.apply(lambda x: sum(float(i) >= data_ap_1['pd'].iloc[min(idx,
len(data_ap_1['pd'])-1)] for idx, i in enumerate(x)))
    data1['sta_to0_above_pd'] = num / total1to0
if 'sta_to_ap_0_max_ant_rssi' in data_ap_0.columns and 'pd' in data_ap_0.columns:
    # 将 vec0to0 中每一行里值大于 pd 的个数赋给 num
    num = vec0to0.apply(lambda x: sum(float(i) >= data_ap_0['pd'].iloc[min(idx,
len(data_ap_0['pd'])-1)] for idx, i in enumerate(x)))
    data0['sta_to0_above_pd'] = num / total0to0
if 'sta_to_ap_1_max_ant_rssi' in data_ap_0.columns and 'pd' in data_ap_0.columns:
    # 将 vec0to1 中每一行里值大于 pd 的个数赋给 num
    num = vec0to1.apply(lambda x: sum(float(i) >= data_ap_0['pd'].iloc[min(idx,
len(data_ap_0['pd'])-1)] for idx, i in enumerate(x)))
    data0['sta_to1_above_pd'] = num / total0to1
if 'sta_to_ap_1_max_ant_rssi' in data_ap_1.columns and 'pd' in data_ap_1.columns:
    # 将 vec1to1 中每一行里值大于 pd 的个数赋给 num
    num = vec1to1.apply(lambda x: sum(float(i) >= data_ap_1['pd'].iloc[min(idx,
len(data_ap_1['pd'])-1)] for idx, i in enumerate(x)))
    data1['sta_to1_above_pd'] = num / total1to1
if 'sta_to_ap_0_max_ant_rssi' in data_ap_1.columns and 'ed' in data_ap_1.columns:
    # 将 vec1to0 中每一行里值大于 ed 的个数赋给 num
    num = vec1to0.apply(lambda x: sum(float(i) >= data_ap_1['ed'].iloc[min(idx,

```



```

len(data_ap_1['ed']-1)] for idx, i in enumerate(x)))
    data1['sta_to0_above_ed'] = num / total1to0
    if 'sta_to_ap_0_max_ant_rssi' in data_ap_0.columns and 'ed' in data_ap_0.columns:
        # 将 vec0to0 中每一行里值大于 ed 的个数赋给 num
        num = vec0to0.apply(lambda x: sum(float(i) >= data_ap_0['ed'].iloc[min(idx,
len(data_ap_0['ed']-1)] for idx, i in enumerate(x)))
        data0['sta_to0_above_ed'] = num / total0to0
    if 'sta_to_ap_1_max_ant_rssi' in data_ap_0.columns and 'ed' in data_ap_0.columns:
        # 将 vec0to1 中每一行里值大于 ed 的个数赋给 num
        num = vec0to1.apply(lambda x: sum(float(i) >= data_ap_0['ed'].iloc[min(idx,
len(data_ap_0['ed']-1)] for idx, i in enumerate(x)))
        data0['sta_to1_above_ed'] = num / total0to1
    if 'sta_to_ap_1_max_ant_rssi' in data_ap_1.columns and 'ed' in data_ap_1.columns:
        # 将 vec1to1 中每一行里值大于 ed 的个数赋给 num
        num = vec1to1.apply(lambda x: sum(float(i) >= data_ap_1['ed'].iloc[min(idx,
len(data_ap_1['ed']-1)] for idx, i in enumerate(x)))
        data1['sta_to1_above_ed'] = num / total1to1

# 将 sta_from_ap_0_max_ant_rssi 的值转化为数组类型
string_0from0 = data_ap_0['sta_from_ap_0_max_ant_rssi'].apply(lambda x: x[1:-1])
vec0from0 = string_0from0.str.split(',')
total0from0 = vec0from0.apply(len)

# 将 sta_from_ap_0_max_ant_rssi 的值转化为数组类型
string_1from0 = data_ap_1['sta_from_ap_0_max_ant_rssi'].apply(lambda x: x[1:-1])
vec1from0 = string_1from0.str.split(',')
total1from0 = vec1from0.apply(len)

# 将 sta_from_ap_1_max_ant_rssi 的值转化为数组类型
string_0from1 = data_ap_0['sta_from_ap_1_max_ant_rssi'].apply(lambda x: x[1:-1])
vec0from1 = string_0from1.str.split(',')
total0from1 = vec0from1.apply(len)

# 将 sta_from_ap_1_max_ant_rssi 的值转化为数组类型
string_1from1 = data_ap_1['sta_from_ap_1_max_ant_rssi'].apply(lambda x: x[1:-1])
vec1from1 = string_1from1.str.split(',')
total1from1 = vec1from1.apply(len)

if 'sta_from_ap_0_max_ant_rssi' in data_ap_1.columns and 'pd' in data_ap_1.columns:
    # 将 vec1from0 中每一行里值大于 pd 的个数赋给 num
    num = vec1from0.apply(lambda x: sum(float(i) >= data_ap_1['pd'].iloc[min(idx,
len(data_ap_1['pd']-1)] for idx, i in enumerate(x)))
    data1['sta_from0_above_pd'] = num / total1from0
    if 'sta_from_ap_0_max_ant_rssi' in data_ap_0.columns and 'pd' in data_ap_0.columns:

```

```

# 将 vec0from0 中每一行里值大于 pd 的个数赋给 num
num = vec0from0.apply(lambda x: sum(float(i) >= data_ap_0['pd'].iloc[min(idx,
len(data_ap_0['pd'])-1)] for idx, i in enumerate(x)))
data0['sta_from0_above_pd'] = num / total0from0
if 'sta_from_ap_1_max_ant_rssi' in data_ap_0.columns and 'pd' in data_ap_0.columns:
# 将 vec0from1 中每一行里值大于 pd 的个数赋给 num
num = vec0from1.apply(lambda x: sum(float(i) >= data_ap_0['pd'].iloc[min(idx,
len(data_ap_0['pd'])-1)] for idx, i in enumerate(x)))
data0['sta_from1_above_pd'] = num / total0from1
if 'sta_from_ap_1_max_ant_rssi' in data_ap_1.columns and 'pd' in data_ap_1.columns:
# 将 vec1from1 中每一行里值大于 pd 的个数赋给 num
num = vec1from1.apply(lambda x: sum(float(i) >= data_ap_1['pd'].iloc[min(idx,
len(data_ap_1['pd'])-1)] for idx, i in enumerate(x)))
data1['sta_from1_above_pd'] = num / total1from1
if 'sta_from_ap_0_max_ant_rssi' in data_ap_1.columns and 'ed' in data_ap_1.columns:
# 将 vec1from0 中每一行里值大于 ed 的个数赋给 num
num = vec1from0.apply(lambda x: sum(float(i) >= data_ap_1['ed'].iloc[min(idx,
len(data_ap_1['ed'])-1)] for idx, i in enumerate(x)))
data1['sta_from0_above_ed'] = num / total1from0
if 'sta_from_ap_0_max_ant_rssi' in data_ap_0.columns and 'ed' in data_ap_0.columns:
# 将 vec0from0 中每一行里值大于 ed 的个数赋给 num
num = vec0from0.apply(lambda x: sum(float(i) >= data_ap_0['ed'].iloc[min(idx,
len(data_ap_0['ed'])-1)] for idx, i in enumerate(x)))
data0['sta_from0_above_ed'] = num / total0from0
if 'sta_from_ap_1_max_ant_rssi' in data_ap_0.columns and 'ed' in data_ap_0.columns:
# 将 vec0from1 中每一行里值大于 ed 的个数赋给 num
num = vec0from1.apply(lambda x: sum(float(i) >= data_ap_0['ed'].iloc[min(idx,
len(data_ap_0['ed'])-1)] for idx, i in enumerate(x)))
data0['sta_from1_above_ed'] = num / total0from1
if 'sta_from_ap_1_max_ant_rssi' in data_ap_1.columns and 'ed' in data_ap_1.columns:
# 将 vec1from1 中每一行里值大于 ed 的个数赋给 num
num = vec1from1.apply(lambda x: sum(float(i) >= data_ap_1['ed'].iloc[min(idx,
len(data_ap_1['ed'])-1)] for idx, i in enumerate(x)))
data1['sta_from1_above_ed'] = num / total1from1

# 将 sta_to_ap_0_mean_ant_rssi 的值转化为数组类型
string_0to0 = data_ap_0['sta_to_ap_0_mean_ant_rssi'].apply(lambda x: x[1:-1])
vec0to0 = string_0to0.str.split(',')
total0to0 = vec0to0.apply(len)

# 将 sta_to_ap_0_mean_ant_rssi 的值转化为数组类型
string_1to0 = data_ap_1['sta_to_ap_0_mean_ant_rssi'].apply(lambda x: x[1:-1])
vec1to0 = string_1to0.str.split(',')
total1to0 = vec1to0.apply(len)

```

```

# 将 sta_to_ap_1_mean_ant_rssi 的值转化为数组类型
string_0to1 = data_ap_0['sta_to_ap_1_mean_ant_rssi'].apply(lambda x: x[1:-1])
vec0to1 = string_0to1.str.split(',')
total0to1 = vec0to1.apply(len)

# 将 sta_to_ap_1_mean_ant_rssi 的值转化为数组类型
string_1to1 = data_ap_1['sta_to_ap_1_mean_ant_rssi'].apply(lambda x: x[1:-1])
vec1to1 = string_1to1.str.split(',')
total1to1 = vec1to1.apply(len)

if 'sta_to_ap_0_mean_ant_rssi' in data_ap_1.columns and 'nav' in data_ap_1.columns:
    # 将 vec1to0 中每一行里值大于 nav 的个数赋给 num
    num = vec1to0.apply(lambda x: sum(float(i) >= data_ap_1['nav'].iloc[min(idx,
len(data_ap_1['nav'])-1)] for idx, i in enumerate(x)))
    data1['sta_to0_above_nav'] = num / total1to0
if 'sta_to_ap_0_mean_ant_rssi' in data_ap_0.columns and 'nav' in data_ap_0.columns:
    # 将 vec0to0 中每一行里值大于 nav 的个数赋给 num
    num = vec0to0.apply(lambda x: sum(float(i) >= data_ap_0['nav'].iloc[min(idx,
len(data_ap_0['nav'])-1)] for idx, i in enumerate(x)))
    data0['sta_to0_above_nav'] = num / total0to0
if 'sta_to_ap_1_mean_ant_rssi' in data_ap_0.columns and 'nav' in data_ap_0.columns:
    # 将 vec0to1 中每一行里值大于 nav 的个数赋给 num
    num = vec0to1.apply(lambda x: sum(float(i) >= data_ap_0['nav'].iloc[min(idx,
len(data_ap_0['nav'])-1)] for idx, i in enumerate(x)))
    data0['sta_to1_above_nav'] = num / total0to1
if 'sta_to_ap_1_mean_ant_rssi' in data_ap_1.columns and 'nav' in data_ap_1.columns:
    # 将 vec1to1 中每一行里值大于 nav 的个数赋给 num
    num = vec1to1.apply(lambda x: sum(float(i) >= data_ap_1['nav'].iloc[min(idx,
len(data_ap_1['nav'])-1)] for idx, i in enumerate(x)))
    data1['sta_to1_above_nav'] = num / total1to1

# 将 sta_from_ap_0_mean_ant_rssi 的值转化为数组类型
string_0from0 = data_ap_0['sta_to_ap_0_mean_ant_rssi'].apply(lambda x: x[1:-1])
vec0from0 = string_0from0.str.split(',')
total0from0 = vec0from0.apply(len)

# 将 sta_from_ap_0_mean_ant_rssi 的值转化为数组类型
string_1from0 = data_ap_1['sta_to_ap_0_mean_ant_rssi'].apply(lambda x: x[1:-1])
vec1from0 = string_1from0.str.split(',')
total1from0 = vec1from0.apply(len)

# 将 sta_from_ap_1_mean_ant_rssi 的值转化为数组类型
string_0from1 = data_ap_0['sta_to_ap_1_mean_ant_rssi'].apply(lambda x: x[1:-1])

```

```

vec0from1 = string_0from1.str.split(',')
total0from1 = vec0from1.apply(len)

# 将 sta_from_ap_1_mean_ant_rssi 的值转化为数组类型
string_1from1 = data_ap_1['sta_to_ap_1_mean_ant_rssi'].apply(lambda x: x[1:-1])
vec1from1 = string_1from1.str.split(',')
total1from1 = vec1from1.apply(len)

if 'sta_from_ap_0_mean_ant_rssi' in data_ap_1.columns and 'nav' in data_ap_1.columns:
    # 将 vec1from0 中每一行里值大于 nav 的个数赋给 num
    num = vec1from0.apply(lambda x: sum(float(i) >= data_ap_1['nav'].iloc[min(idx,
len(data_ap_1['nav'])-1)] for idx, i in enumerate(x)))
    data1['sta_from0_above_nav'] = num / total1from0
if 'sta_from_ap_0_mean_ant_rssi' in data_ap_0.columns and 'nav' in data_ap_0.columns:
    # 将 vec0from0 中每一行里值大于 nav 的个数赋给 num
    num = vec0from0.apply(lambda x: sum(float(i) >= data_ap_0['nav'].iloc[min(idx,
len(data_ap_0['nav'])-1)] for idx, i in enumerate(x)))
    data0['sta_from0_above_nav'] = num / total0from0
if 'sta_from_ap_1_mean_ant_rssi' in data_ap_0.columns and 'nav' in data_ap_0.columns:
    # 将 vec0from1 中每一行里值大于 nav 的个数赋给 num
    num = vec0from1.apply(lambda x: sum(float(i) >= data_ap_0['nav'].iloc[min(idx,
len(data_ap_0['nav'])-1)] for idx, i in enumerate(x)))
    data0['sta_from1_above_nav'] = num / total0from1
if 'sta_from_ap_1_mean_ant_rssi' in data_ap_1.columns and 'nav' in data_ap_1.columns:
    # 将 vec1from1 中每一行里值大于 nav 的个数赋给 num
    num = vec1from1.apply(lambda x: sum(float(i) >= data_ap_1['nav'].iloc[min(idx,
len(data_ap_1['nav'])-1)] for idx, i in enumerate(x)))
    data1['sta_from1_above_nav'] = num / total1from1
return data0, data1
# Example usage
if __name__ == "__main__":
    file_path = 'training_set_2ap_loc0_nav82.csv'
    data0,data1 = read_and_display_2ap_csv(file_path)
    # 输出前 5 行数据
    print(data0.head())
    print(data1.head())

```

附录 12: randomForest.py

```

import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

```

```

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt
from readData import read_and_display_2ap_csv
from readAllData import read_all_data

# 读取 CSV 文件
file_path = 'training_set_2ap_loc0_nav82.csv'
data0, data1 = read_and_display_2ap_csv(file_path)

# 合并 data0 和 data1
# data = pd.concat([data0, data1], axis=0)

data = read_all_data()

# 信号强度转换为线性
data['pd'] = 10 ** (data['pd'] / 10)
data['ed'] = 10 ** (data['ed'] / 10)
data['nav'] = 10 ** (data['nav'] / 10)

# 提取特征和目标变量
X = data.drop(columns=['seq_time', 'throughput', 'other_air_time', 'ppdu_dur', 'num_ampdu', 'per', 'mcs',
'nss'])
y = data['seq_time']

# 处理缺失值
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# 标准化数据
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# 训练随机森林模型
model = RandomForestRegressor(random_state=42)
model.fit(X_train, y_train)

# 预测
y_pred = model.predict(X_test)

```

```

# 计算评估指标
mse = mean_squared_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"Mean Absolute Error (MAE): {mae}")
print(f"R^2 Score: {r2}")

# 分析特征重要性
importance = model.feature_importances_
importance_df = pd.DataFrame({'Feature': X.columns, 'Importance': importance})
importance_df = importance_df.sort_values(by='Importance', ascending=False)

print("Feature importance:")
print(importance_df)

# 可视化特征重要性
plt.figure(figsize=(10, 6))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.title('Feature Importance for seq_time')
plt.gca().invert_yaxis()
plt.show()

```

附录 13: pca.py

```

import pandas as pd
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
import matplotlib.pyplot as plt
from readData import read_and_display_2ap_csv
from readAllData import read_all_data

# 读取并合并所有数据
data = read_all_data()

# 选择特征列（去除非数值列和目标列）
features = data.drop(columns=['nss', 'mcs', 'per', 'num_ampdu', 'throughput', 'ppdu_dur', 'seq_time',

```

```

'other_air_time'])

# 打印特征列
print("Original features shape:", features.shape)

# 处理缺失值
imputer = SimpleImputer(strategy='mean')
features_imputed = imputer.fit_transform(features)

# 打印处理缺失值后的形状
print("Features shape after imputation:", features_imputed.shape)

# 标准化数据
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features_imputed)

# 进行 PCA 分析
pca = PCA(n_components=2) # 选择前两个主成分
principal_components = pca.fit_transform(features_scaled)

# 创建包含主成分的数据框
principal_df = pd.DataFrame(data=principal_components, columns=['principal_component_1',
'principal_component_2'])

# 打印 PCA 结果
print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Principal components shape:", principal_df.shape)

# 计算特征重要度（主成分载荷）
loadings = pca.components_.T * np.sqrt(pca.explained_variance_)
feature_importance = pd.DataFrame(loadings, index=features.columns, columns=['PC1', 'PC2'])

# 计算每个特征的总重要度
feature_importance['Total'] = feature_importance.abs().sum(axis=1)

# 对特征重要度进行排序
sorted_feature_importance = feature_importance.sort_values(by='Total', ascending=False)

print("Sorted feature importance:")
print(sorted_feature_importance)

# 可视化 PCA 结果
plt.figure(figsize=(10, 6))
plt.scatter(principal_df['principal_component_1'], principal_df['principal_component_2'],

```

```

alpha=0.5)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA of 2AP Data')
plt.show()

```

附录 14: pairing.py

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from readData import read_and_display_2ap_csv
from readAllData import read_all_data

# 读取并合并所有数据
data = read_all_data()

# 选择特征列（去除非数值列和目标列）
features = data.drop(columns=['nss', 'mcs', 'per', 'num_ampdu', 'throughput', 'ppdu_dur', 'other_air_time'])
features = features.drop(columns=['test_dur', 'pkt_len', 'pd', 'ed', 'nav'])
# 计算相关矩阵
correlation_matrix = features.corr()

# 打印相关矩阵
print("Correlation matrix:")
print(correlation_matrix)

# 设置绘图风格
sns.set(style='white')

# 创建一个自定义的颜色映射
cmap = sns.diverging_palette(220, 20, as_cmap=True)

# 可视化相关矩阵
plt.figure(figsize=(14, 12))
heatmap = sns.heatmap(correlation_matrix, annot=True, cmap=cmap, fmt='.2f', linewidths=0.5,
                      cbar_kws={"shrink": 0.75}, vmin=-1, vmax=1, center=0, square=True,
                      annot_kws={"size": 10})

# 设置标题和轴标签
plt.title('2AP feature correlation heatmap', fontsize=18)
plt.xticks(rotation=45, ha='right', fontsize=12)
plt.yticks(fontsize=12)

```



```
plt.tight_layout()
```

```
# 显示图表
```

```
plt.show()
```

附录 15: cnn_predict3.py

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.impute import SimpleImputer
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from torch.utils.data import DataLoader, TensorDataset
```

```
from readAllData import read_all_data
```

```
from readData import read_and_display_2ap_csv
```

```
from readData2 import map_nss_mcs2
```

```
# 读取 CSV 文件
```

```
data = read_all_data()
```

```
# 提取特征和目标变量
```

```
X = data.drop(columns=['throughput', 'other_air_time', 'ppdu_dur', 'num_ampdu'])
```

```
X['target_PHY_rate'] = X.apply(lambda row: map_nss_mcs2(row['nss'], row['mcs']), axis=1)
```

```
# 删除非数值列
```

```
X = X.select_dtypes(include=[np.number])
```

```
y = data['throughput']
```

```
# 处理缺失值
```

```
imputer = SimpleImputer(strategy='mean')
```

```
X_imputed = imputer.fit_transform(X)
```

```
# 标准化数据
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X_imputed)
```

```
# 将数据调整为 CNN 输入格式
```

```
X_scaled = np.expand_dims(X_scaled, axis=2)
```

```

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.01, random_state=42)

# 转换为 PyTorch 张量
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32).unsqueeze(1)
y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32).unsqueeze(1)

# 调整张量的形状
X_train_tensor = X_train_tensor.permute(0, 2, 1)
X_test_tensor = X_test_tensor.permute(0, 2, 1)

# 创建数据加载器
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# 计算特征图大小
def calculate_output_size(input_size, kernel_size, stride=1, padding=0):
    return (input_size - kernel_size + 2 * padding) // stride + 1

# 输入大小
input_size = X_train.shape[1]

# 经过第一个卷积层和池化层后的大小
conv1_output_size = calculate_output_size(input_size, kernel_size=2)
pool1_output_size = calculate_output_size(conv1_output_size, kernel_size=2, stride=2)

# 经过第二个卷积层和池化层后的大小
conv2_output_size = calculate_output_size(pool1_output_size, kernel_size=2)
pool2_output_size = calculate_output_size(conv2_output_size, kernel_size=2, stride=2)

# 定义更复杂的 CNN 模型
class CNNRegression(nn.Module):
    def __init__(self):
        super(CNNRegression, self).__init__()
        self.conv1 = nn.Conv1d(in_channels=1, out_channels=64, kernel_size=2)
        self.conv2 = nn.Conv1d(in_channels=64, out_channels=128, kernel_size=2)
        self.pool = nn.MaxPool1d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(128 * pool2_output_size, 100)
        self.fc2 = nn.Linear(100, 50)
        self.fc3 = nn.Linear(50, 1)

```

```

def forward(self, x):
    x = torch.relu(self.conv1(x))
    x = self.pool(x)
    x = torch.relu(self.conv2(x))
    x = self.pool(x)
    x = x.reshape(x.size(0), -1)
    x = torch.relu(self.fc1(x))
    x = torch.relu(self.fc2(x))
    x = self.fc3(x)
    return x

# 初始化模型、损失函数和优化器
model = CNNRegression()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 训练模型
num_epochs = 200
for epoch in range(num_epochs):
    model.train()
    total_error = 0
    num_batches = 0
    all_errors = []
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        # 计算 outputs 和 y_batch 之间的误差百分比
        batch_error = torch.abs((outputs - y_batch) / y_batch)
        total_error += batch_error.mean().item()
        num_batches += 1
        all_errors.extend(batch_error.detach().numpy().flatten()) # 使用 detach()分离计算图

# 将所有误差排序
all_errors = np.array(all_errors)
sorted_errors = np.sort(all_errors)

# 取排在前十10%的那一个误差
threshold_index = int(len(sorted_errors) * 0.1)
top_10_percent_error = sorted_errors[threshold_index]

```

```

acc = 1-top_10_percent_error

avg_error = total_error / num_batches

print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}, Avg Error: {avg_error:.2f} ,
Accuracy: {acc:.2f}')
with open('loss_accuracy3_2ap.csv', 'a') as f:
    f.write(f'{epoch+1},{loss.item()},{avg_error},{acc}\n')

# 评估模型
model.eval()
with torch.no_grad():
    y_pred = model(X_test_tensor)
    test_loss = criterion(y_pred, y_test_tensor)
    print(f'Test Loss: {test_loss.item():.4f}')

# 打印预测结果
print("Predicted seq_time:", y_pred.numpy().flatten())
print("Actual seq_time:", y_test_tensor.numpy().flatten())

# 保存模型
torch.save(model.state_dict(), 'cnn_model3.pth')
print("Model saved as cnn_model.pth")

# 读取 CSV 文件
file_path = 'test_set_1_2ap.csv'
data0,data1 = read_and_display_2ap_csv(file_path)
data = pd.concat([data0, data1], axis=0).sort_index()
data3 = pd.read_csv('predicted_test_set_1_2ap.csv')
# 提取特征
X = data.drop(columns=['throughput', 'other_air_time', 'ppdu_dur', 'num_ampdu'])
X = X.drop(columns=['error%', 'error%.1', 'predict seq_time', 'predict throughput'])
X['seq_time'] = data3['predicted_seq_time']
X['target_PHY_rate'] = X.apply(lambda row: map_nss_mcs2(row['nss'], row['mcs']), axis=1)

# 输入模型进行预测
X_imputed = imputer.transform(X)
X_scaled = scaler.transform(X_imputed)
X_scaled = np.expand_dims(X_scaled, axis=2)
X_tensor = torch.tensor(X_scaled, dtype=torch.float32)
X_tensor = X_tensor.permute(0, 2, 1)
with torch.no_grad():
    y_pred = model(X_tensor)

```

```
# 将预测结果添加到原始数据中
data['predicted_throughput'] = y_pred.numpy().flatten()

# 保存结果到新的 CSV 文件
#output_file_path = 'predicted_test_set_1_2ap3.csv'
data.to_csv(output_file_path, index=False)
print(f"Predictions saved to {output_file_path}")
```

附录 16: cnn_predict2_v2.py

```
from readAllData import read_all_data
from readData import read_and_display_2ap_csv
from readData2 import read_2ap_for_question2
from readAllData import read_all_data_for_q2
from readData2 import map_nss_mcs2
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.metrics import accuracy_score
from readData2 import guess_nss_mcs

# 读取 CSV 文件
data = read_all_data()
data2 = read_all_data_for_q2()
# 提取特征和目标变量
data2['id_0'] = data2['id_0'].apply(lambda x: np.repeat(x, 540))
data2['id_1'] = data2['id_1'].apply(lambda x: np.repeat(x, 540))
# 使用 map_nss_mcs 将 nss 和 mcs 特征转换为 target_PHY_rate
data['target_PHY_rate'] = data.apply(lambda row: map_nss_mcs2(row['nss'], row['mcs']), axis=1)

# data2 中的特征都是长度为 540 的列表，需要将其转换为矩阵
matrices = [np.stack(lst, axis=0) for lst in data2.values]
# 将这些矩阵转化为最终的 392 个矩阵
final_matrices = np.array(matrices)
print(final_matrices.shape) # 输出形状以验证结果
X = final_matrices
y = data['target_PHY_rate'].values
```

```

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 转换为 PyTorch 张量
X_train_tensor = torch.tensor(X_train, dtype=torch.float32).unsqueeze(1)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32).unsqueeze(1)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

# 定义二维卷积神经网络模型
class CNNRegression2D(nn.Module):
    def __init__(self):
        super(CNNRegression2D, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=64, kernel_size=(3, 3), padding=1)
        self.conv2 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=(3, 3), padding=1)
        self.pool = nn.MaxPool2d(kernel_size=(2, 2), stride=(2, 2))

        # 计算池化层之后的张量形状
        self._to_linear = None
        self.convs(torch.randn(1, 1, 6, 540)) # 通过一个虚拟的输入来计算

        self.fc1 = nn.Linear(self._to_linear, 100)
        self.fc2 = nn.Linear(100, 50)
        self.fc3 = nn.Linear(50, 1)

    def convs(self, x):
        x = torch.relu(self.conv1(x))
        x = self.pool(x)
        x = torch.relu(self.conv2(x))
        x = self.pool(x)
        if self._to_linear is None:
            self._to_linear = x[0].numel()
        return x

    def forward(self, x):
        x = self.convs(x)
        x = x.reshape(x.size(0), -1)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        #x = torch.softmax(x, dim=1) # 添加 Softmax 层
        return x

```

```

# 初始化模型、损失函数和优化器
model = CNNRegression2D()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.002)

# 训练模型
num_epochs = 500
for epoch in range(num_epochs):
    model.train()
    optimizer.zero_grad()
    outputs = model(X_train_tensor)
    loss = criterion(outputs, y_train_tensor)
    loss.backward()
    optimizer.step()
    # 计算准确率
    correct = 0
    total = y_test_tensor.size(0)
    for i in range(total):
        true_nss, true_mcs = guess_nss_mcs(y_test_tensor[i].item())
        pred_nss, pred_mcs = guess_nss_mcs(outputs[i].item())
        if true_nss == pred_nss and true_mcs == pred_mcs:
            correct += 1
    accuracy = correct / total
    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}, Accuracy: {accuracy}')
    # 保存 loss 和 accuracy 到文件
    with open('loss_accuracy2v2.csv', 'a') as f:
        f.write(f'{epoch+1},{loss.item()},{accuracy}\n')

# 评估模型
model.eval()
with torch.no_grad():
    y_pred = model(X_test_tensor)
    test_loss = criterion(y_pred, y_test_tensor)
    # 计算准确率
    correct = 0
    total = y_test_tensor.size(0)
    for i in range(total):
        true_nss, true_mcs = guess_nss_mcs(y_test_tensor[i].item())
        pred_nss, pred_mcs = guess_nss_mcs(y_pred[i].item())
        if true_nss == pred_nss and true_mcs == pred_mcs:
            correct += 1
    accuracy = correct / total
    print(f'Test Loss: {test_loss.item():.4f}, Accuracy: {accuracy}')

```

```

# 输出测试结果
y_pred = y_pred.squeeze()
y_test_tensor = y_test_tensor.squeeze()
print("Predicted values:")
print(y_pred)
print("Actual values:")
print(y_test_tensor)

# 保存模型
torch.save(model.state_dict(), 'cnn_model_2d.pth')
print("Model saved as cnn_model_2d.pth")

data3 = read_2ap_for_question2('test_set_2_2ap.csv')
print(data3)
data3 = data3.drop(columns=['predict nss', 'predict mcs', 'error%'])
data3['id_0'] = data3['id_0'].apply(lambda x: np.repeat(x, 540))
data3['id_1'] = data3['id_1'].apply(lambda x: np.repeat(x, 540))
test_matrices = [np.stack(lst, axis=0) for lst in data3.values]
test_final_matrices = np.array(test_matrices)
X_test = test_final_matrices
X_test_tensor = torch.tensor(X_test, dtype=torch.float32).unsqueeze(1)
with torch.no_grad():
    y_pred = model(X_test_tensor)
y_pred = y_pred.squeeze()
data3['predicted_target_PHY_rate'] = y_pred.numpy()
output_file_path = 'predicted_test_set_2_2ap.csv'
data3.to_csv(output_file_path, index=False)
print(f'Predictions saved to {output_file_path}')

```

附录 17: cnn_predict2.py

```

from readAllData import read_all_data
from readData import read_and_display_2ap_csv
from readData2 import read_2ap_for_question2
from readAllData import read_all_data_for_q2
from readData2 import map_nss_mcs
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import torch
import torch.nn as nn
import torch.optim as optim

```



```

from sklearn.metrics import accuracy_score
import pandas as pd

# 读取 CSV 文件
data = read_all_data()
data2 = read_all_data_for_q2()
# 提取特征和目标变量
data2['id_0'] = data2['id_0'].apply(lambda x: np.repeat(x, 540))
data2['id_1'] = data2['id_1'].apply(lambda x: np.repeat(x, 540))
# 使用 map_nss_mcs 将 nss 和 mcs 特征转换为 target_PHY_rate
data['target_PHY_rate'] = data.apply(lambda row: map_nss_mcs(row['nss'], row['mcs']), axis=1)

# data2 中的特征都是长度为 540 的列表，需要将其转换为矩阵
matrices = [np.stack(lst, axis=0) for lst in data2.values]
# 将这些矩阵转化为最终的 392 个矩阵
final_matrices = np.array(matrices)
print(final_matrices.shape) # 输出形状以验证结果
X = final_matrices
y = data['target_PHY_rate'].values

y = np.array(y.tolist(), dtype=np.float32)

print(type(y))
print(y.shape)
print(y[:5])
# 将独热编码转换为类别索引
# y_indices = np.array([np.argmax(label) for label in y])
# 输出 y 为 csv 文件
# y.to_csv('y.csv', index=False)

# 标准化数据
# scaler = StandardScaler()
# X = scaler.fit_transform(X.reshape(-1, X.shape[-1])).reshape(X.shape)

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 转换为 PyTorch 张量
X_train_tensor = torch.tensor(X_train, dtype=torch.float32).unsqueeze(1)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32).unsqueeze(1)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)

```

定义二维卷积神经网络模型

```
class CNNRegression2D(nn.Module):
```

```
    def __init__(self):
```

```
        super(CNNRegression2D, self).__init__()
```

```
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=64, kernel_size=(3, 3),padding=1)
```

```
        self.conv2 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=(3, 3),padding=1)
```

```
        self.pool = nn.MaxPool2d(kernel_size=(2, 2), stride=(2, 2))
```

```
        # 计算池化层之后的张量形状
```

```
        self.to_linear = None
```

```
        self.convs(torch.randn(1, 1, 6, 540)) # 通过一个虚拟的输入来计算
```

```
        self.fc1 = nn.Linear(self.to_linear, 100)
```

```
        self.fc2 = nn.Linear(100, 50)
```

```
        self.fc3 = nn.Linear(50, 19)
```

```
    def convs(self, x):
```

```
        x = torch.relu(self.conv1(x))
```

```
        x = self.pool(x)
```

```
        x = torch.relu(self.conv2(x))
```

```
        x = self.pool(x)
```

```
        if self.to_linear is None:
```

```
            self.to_linear = x[0].numel()
```

```
        return x
```

```
    def forward(self, x):
```

```
        x = self.convs(x)
```

```
        x = x.reshape(x.size(0), -1)
```

```
        x = torch.relu(self.fc1(x))
```

```
        x = torch.relu(self.fc2(x))
```

```
        x = self.fc3(x)
```

```
        x = torch.softmax(x, dim=1) # 添加 Softmax 层
```

```
        return x
```

初始化模型、损失函数和优化器

```
model = CNNRegression2D()
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

训练模型

```
num_epochs = 20
```

```
for epoch in range(num_epochs):
```

```
    model.train()
```

```
    optimizer.zero_grad()
```

```

outputs = model(X_train_tensor)
loss = criterion(outputs, y_train_tensor)
loss.backward()
optimizer.step()
print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')
y_pred_indices = torch.argmax(outputs, dim=1)
y_train_indices = torch.argmax(y_train_tensor, dim=1)
accuracy = accuracy_score(y_pred_indices.numpy(), y_train_indices.numpy())
print(f'Accuracy: {accuracy:.4f}')
# 保存 loss 和 accuracy 到文件
with open('loss_accuracy2.csv', 'a') as f:
    f.write(f'{epoch+1},{loss.item()},{accuracy}\n')

# 评估模型
model.eval()
with torch.no_grad():
    y_pred = model(X_test_tensor)
    test_loss = criterion(y_pred, y_test_tensor)
    print(f'Test Loss: {test_loss.item():.4f}')

# 将 y_pred 和 y_test_tensor 转换为类别索引
y_pred_indices = torch.argmax(y_pred, dim=1)
y_test_indices = torch.argmax(y_test_tensor, dim=1)

# 计算准确率
accuracy = accuracy_score(y_test_indices.numpy(), y_pred_indices.numpy())
print(f'Accuracy: {accuracy:.4f}')

# 将 y_pred 和 y_test_tensor 转为整数
y_pred = torch.argmax(y_pred, dim=1)
y_test_tensor = torch.argmax(y_test_tensor, dim=1)
# 将 y_pred 和 y_test_tensor 转换为 pandas DataFrame
y_pred_df = pd.DataFrame(y_pred.numpy(), columns=['predicted'])
y_test_tensor_df = pd.DataFrame(y_test_tensor.numpy(), columns=['actual'])

# 保存到 CSV 文件
y_pred_df.to_csv('y_pred.csv', index=False)
y_test_tensor_df.to_csv('y_test_tensor.csv', index=False)
# 保存模型
torch.save(model.state_dict(), 'cnn_model_2d.pth')
print("Model saved as cnn_model_2d.pth")

```

附录 18: cnn_predict1.py

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from readAllData import read_all_data
from readData import read_and_display_2ap_csv

# 读取 CSV 文件
data = read_all_data()

# 提取特征和目标变量
X = data.drop(columns=['seq_time', 'throughput', 'other_air_time', 'ppdu_dur', 'num_ampdu', 'per',
'mcs', 'nss'])

# 删除非数值列
X = X.select_dtypes(include=[np.number])

y = data['seq_time']

# 处理缺失值
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# 标准化数据
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)

# 将数据调整为 CNN 输入格式
X_scaled = np.expand_dims(X_scaled, axis=2)

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.01, random_state=42)

# 转换为 PyTorch 张量
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32).unsqueeze(1)
```

```

y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32).unsqueeze(1)

# 调整张量的形状
X_train_tensor = X_train_tensor.permute(0, 2, 1)
X_test_tensor = X_test_tensor.permute(0, 2, 1)

# 创建数据加载器
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# 计算特征图大小
def calculate_output_size(input_size, kernel_size, stride=1, padding=0):
    return (input_size - kernel_size + 2 * padding) // stride + 1

# 输入大小
input_size = X_train.shape[1]

# 经过第一个卷积层和池化层后的大小
conv1_output_size = calculate_output_size(input_size, kernel_size=2)
pool1_output_size = calculate_output_size(conv1_output_size, kernel_size=2, stride=2)

# 经过第二个卷积层和池化层后的大小
conv2_output_size = calculate_output_size(pool1_output_size, kernel_size=2)
pool2_output_size = calculate_output_size(conv2_output_size, kernel_size=2, stride=2)

# 定义更复杂的 CNN 模型
class CNNRegression(nn.Module):
    def __init__(self):
        super(CNNRegression, self).__init__()
        self.conv1 = nn.Conv1d(in_channels=1, out_channels=64, kernel_size=2)
        self.conv2 = nn.Conv1d(in_channels=64, out_channels=128, kernel_size=2)
        self.pool = nn.MaxPool1d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(128 * pool2_output_size, 100)
        self.fc2 = nn.Linear(100, 50)
        self.fc3 = nn.Linear(50, 1)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.pool(x)
        x = torch.relu(self.conv2(x))
        x = self.pool(x)
        x = x.reshape(x.size(0), -1)

```

```

        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x

# 初始化模型、损失函数和优化器
model = CNNRegression()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 训练模型
num_epochs = 100
for epoch in range(num_epochs):
    model.train()
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')

# 评估模型
model.eval()
with torch.no_grad():
    y_pred = model(X_test_tensor)
    test_loss = criterion(y_pred, y_test_tensor)
    print(f'Test Loss: {test_loss.item():.4f}')

# 打印预测结果
print("Predicted seq_time:", y_pred.numpy().flatten())
print("Actual seq_time:", y_test_tensor.numpy().flatten())

# 保存模型
torch.save(model.state_dict(), 'cnn_model.pth')
print("Model saved as cnn_model.pth")

# 读取 CSV 文件
file_path = 'test_set_1_2ap.csv'
data0, data1 = read_and_display_2ap_csv(file_path)
data = pd.concat([data0, data1], axis=0).sort_index()

# 提取特征
X = data.drop(columns=['seq_time', 'throughput', 'other_air_time', 'ppdu_dur', 'num_ampdu', 'per',

```

```

'mcs', 'nss'])
X = X.drop(columns=['error%', 'error%.1', 'predict seq_time', 'predict throughput'])

# 输入模型进行预测
X_imputed = imputer.transform(X)
X_scaled = scaler.transform(X_imputed)
X_scaled = np.expand_dims(X_scaled, axis=2)
X_tensor = torch.tensor(X_scaled, dtype=torch.float32)
X_tensor = X_tensor.permute(0, 2, 1)
with torch.no_grad():
    y_pred = model(X_tensor)

# 将预测结果添加到原始数据中
data['predicted_seq_time'] = y_pred.numpy().flatten()

# 保存结果到新的 CSV 文件
output_file_path = 'predicted_test_set_1_2ap.csv'
data.to_csv(output_file_path, index=False)
print(f'Predictions saved to {output_file_path}')

```

附录 19: cnn_predict_for_3ap3.py

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# 读取 CSV 文件
train_data = pd.read_csv('./output/07.standardized_combined_3ap.csv')
print(train_data.head())
test_data = pd.read_csv('./output/08.separated_test_set_3ap.csv')
print(test_data.head())

# 提取特征和目标变量
X_train = train_data.drop(columns=['throughput'])
y_train = train_data['throughput']
X_test = test_data.drop(columns=['throughput'])
y_test = test_data['throughput']

```

```

data2 = pd.read_csv('./output/09.predicted_test_set_3ap-1.csv')
X_test['seq_time'] = data2['seq_time']

# 处理缺失值
imputer = SimpleImputer(strategy='mean')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# 将数据调整为 CNN 输入格式
X_train_scaled = np.expand_dims(X_train_imputed, axis=2)
X_test_scaled = np.expand_dims(X_test_imputed, axis=2)

# 转换为 PyTorch 张量
X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32).unsqueeze(1)
y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32).unsqueeze(1)

# 调整张量的形状
X_train_tensor = X_train_tensor.permute(0, 2, 1)
X_test_tensor = X_test_tensor.permute(0, 2, 1)

# 创建数据加载器
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# 计算特征图大小
def calculate_output_size(input_size, kernel_size, stride=1, padding=0):
    return (input_size - kernel_size + 2 * padding) // stride + 1

# 输入大小
input_size = X_train_scaled.shape[1]

# 经过第一个卷积层和池化层后的大小
conv1_output_size = calculate_output_size(input_size, kernel_size=2)
pool1_output_size = calculate_output_size(conv1_output_size, kernel_size=2, stride=2)

# 经过第二个卷积层和池化层后的大小
conv2_output_size = calculate_output_size(pool1_output_size, kernel_size=2)
pool2_output_size = calculate_output_size(conv2_output_size, kernel_size=2, stride=2)

```



```

class CNNRegression(nn.Module):
    def __init__(self):
        super(CNNRegression, self).__init__()
        self.conv1 = nn.Conv1d(in_channels=1, out_channels=64, kernel_size=2)
        self.conv2 = nn.Conv1d(in_channels=64, out_channels=128, kernel_size=2)
        self.pool = nn.MaxPool1d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(128 * pool2_output_size, 100)
        self.fc2 = nn.Linear(100, 50)
        self.fc3 = nn.Linear(50, 1)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.pool(x)
        x = torch.relu(self.conv2(x))
        x = self.pool(x)
        x = x.reshape(x.size(0), -1)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x

# 创建模型、损失函数和优化器
model = CNNRegression()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 训练模型
num_epochs = 200
for epoch in range(num_epochs):
    model.train()
    total_error = 0
    num_batches = 0
    all_errors = []
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        # 计算 outputs 和 y_batch 之间的误差百分比
        batch_error = torch.abs((outputs - y_batch) / y_batch)
        total_error += batch_error.mean().item()
        num_batches += 1
    all_errors.extend(batch_error.detach().numpy().flatten()) # 使用 detach()分离计算图

```

```

# 将所有误差排序
all_errors = np.array(all_errors)
sorted_errors = np.sort(all_errors)

# 取排在前 10%的那一个误差
threshold_index = int(len(sorted_errors) * 0.1)
top_10_percent_error = sorted_errors[threshold_index]

acc = 1-top_10_percent_error

avg_error = total_error / num_batches
print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')
with open('loss_accuracy3_3ap.csv', 'a') as f:
    f.write(f'{epoch+1},{loss.item()},{avg_error},{acc}\n')

# 评估模型
model.eval()
with torch.no_grad():
    y_pred = model(X_test_tensor)
    test_loss = criterion(y_pred, y_test_tensor)
    print(f'Test Loss: {test_loss.item():.4f}')

data2['throughput'] = y_pred.numpy()
data2.to_csv('./output/10.predicted_test_set_3ap.csv', index=False)

# 保存模型
torch.save(model.state_dict(), './output/cnn_regression_model.pth')
print("Model saved to cnn_regression_model.pth")

```